Class-02 **Software Engineering** 23/07/2025

Engineering → Efficiency
Scientific Methodology
Services/Products.
Systematic way
executable

Program → Set of ^instructions to solve a ~~progr~~ problem.
(Code, UI) <programmers>

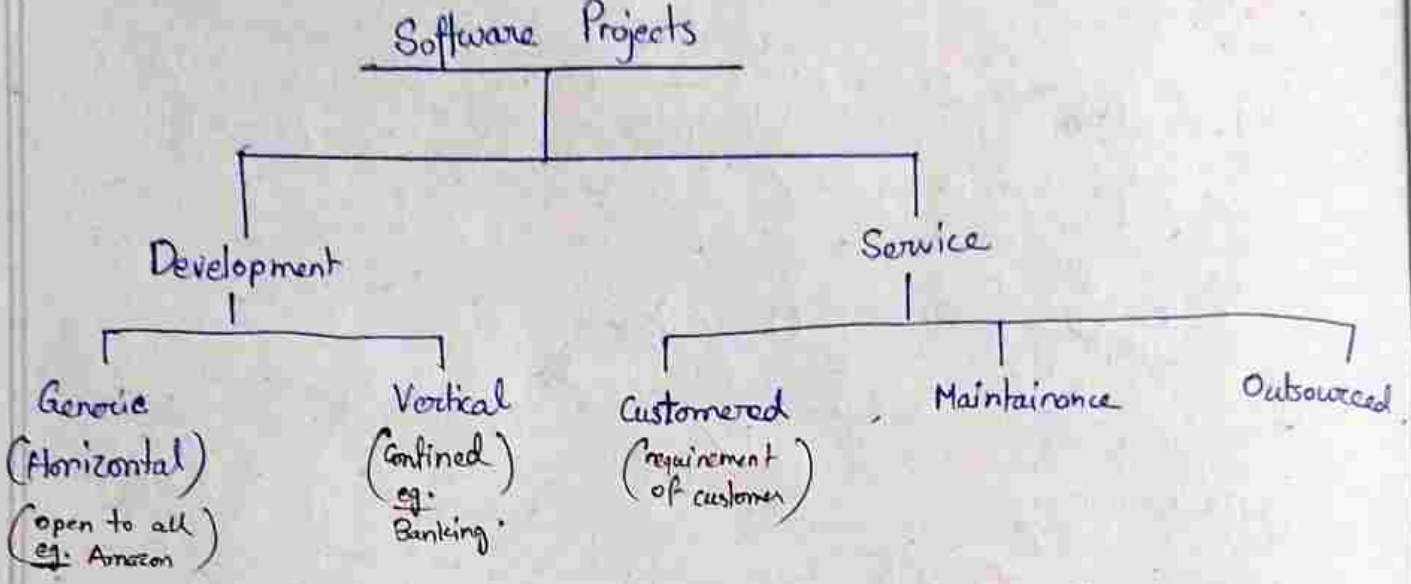
Software → Set of programs to solve a real world problem or to exhibit automation.
(Code | UI |
<Users | Programmers>
↳ more than program / less than product

Software Product → Finest outcome of software.
(UI | User Manual | Proper Requirement Analysis | Estimation

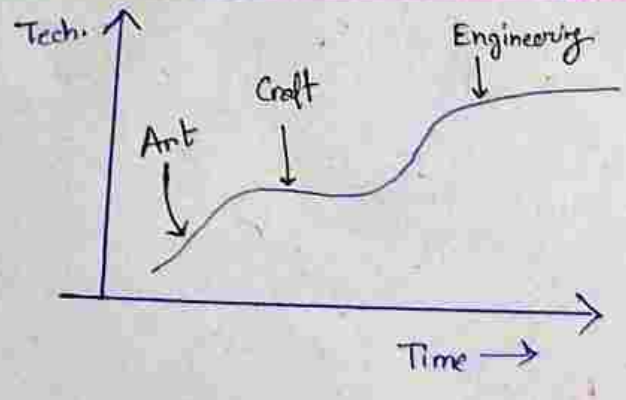
An engineering approach to develop a software. It should be systematic, with —
- Requirement Analysis
- Estimation (Cost of developing | Time of developing | line of codes)
- Design
- Develop
- Testing
- Deployment
- Maintainance (Product part)

Class-10 Software Engineering 01/08/2025

Types of Software Projects



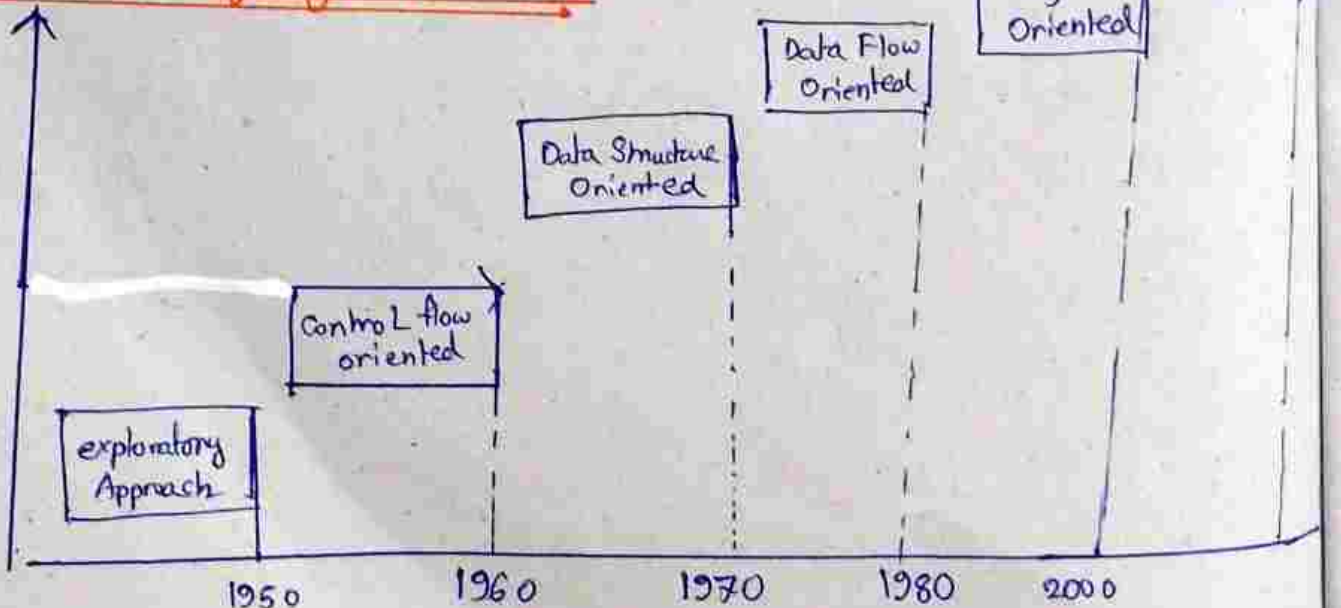
Evolution of S.E. via technology



Design & Analysis of Algorithm
- flow chart
- architecture
- how can we find sol'

→ step by step instruction to solve a proble

Evolution of Designing Approach



- Data Structure → how to organise data
  → which of function you can apply upon those dat

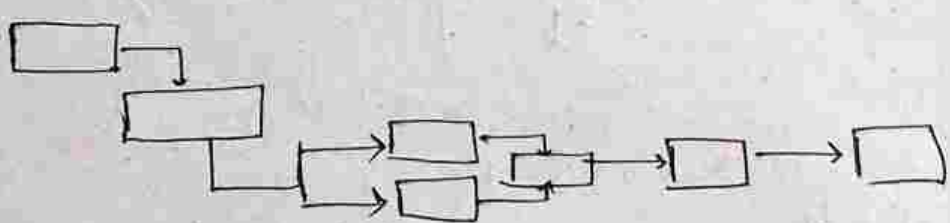
Computer Systems Engineering

- ~~Engine~~ A systematic way of designing computer systems focusing upon —
- Designing Hardware, • Software designing and ~~Interrelation~~ Integration of the

Steps / Phases of Project Management

1. Feasibility Study & Specification.
2. Requirement Analysis
3. Hardware Development + Software Development.
4. Integration
5. Testing
6. Maintainance

(can show in diagram like this.)



H/W

1. Difference between process and methodology

Life Cycle of Software :

> Life cycle of a software represents series of identifiable stages through which the software evolves during its lifespan.

Software Development Life Cycle Model : (SDLC)

- The software engineering approaches emphasize software development ~~to the~~ through a well defined and ordered set of activities,
- When these activities are graphically modeled and textually described are called software life cycle model or SDLC Model or Software Development Process Model.

Process vs Methodology

Process
- The process describes all the activities starting from its inception to maintainance and retirement of the software.
- It is a broader concept.

Methodology
> It describes steps to carry out only a single or atleast a few individual activities.
> It is a narrower concept. (detailed/specific)

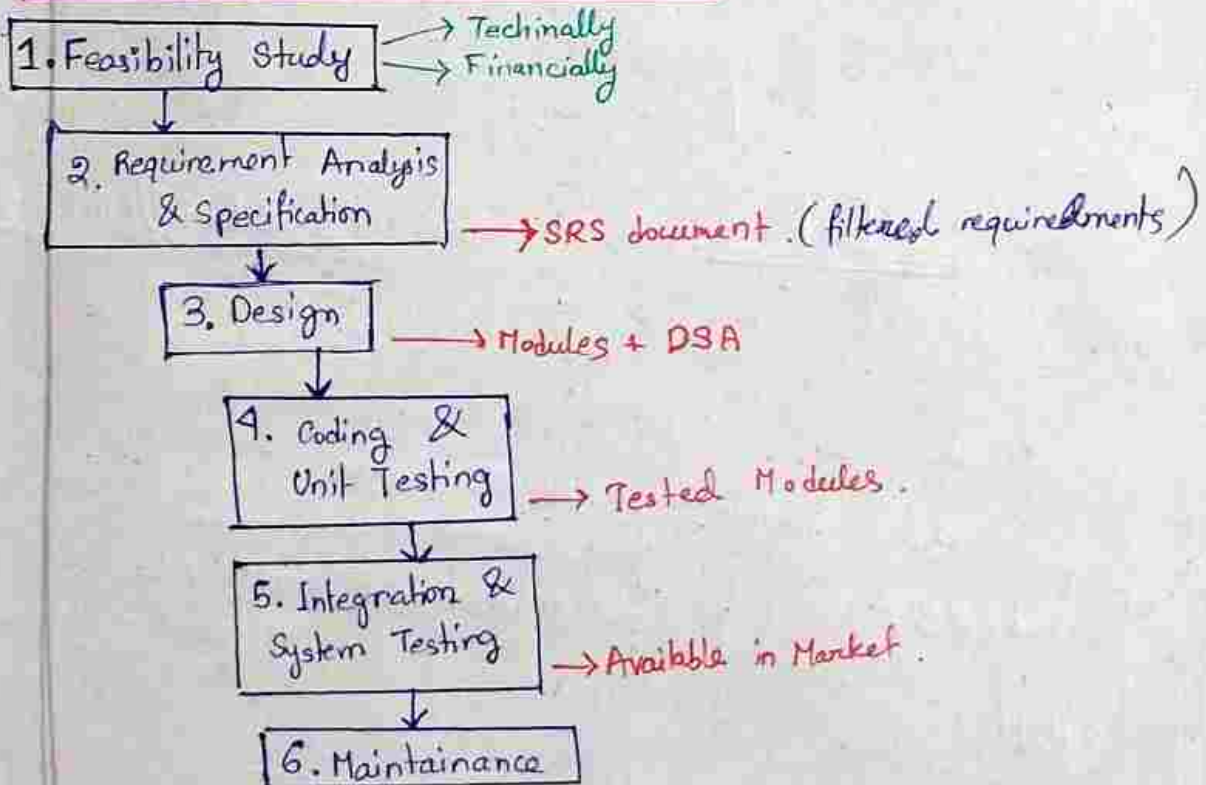
99% Complete Syndrome :

- The identified stages in SDLC should be unambigious, and should have the entry and exit criteria.
- The entry or exit criteria is usually expressed as a set of conditions that needs to be satisfied for the phase to start or to complete.
- After these criteria are satisfied, the next phase can start.

23
19
17

- When the phase entry and exit criteria are not well defined, the developers might close the activities of a phase much before they are actually complete, giving a false impression of rapid progress.
- It is very difficult for the project manager to determine the exact status of the development, and track the progress of a project; This usually leads to a problem identified as 99% complete syndrome.

Classical Waterfall Model :



Order of time/duration : 6 > ~~4~~ (4,5) > 3 > 2 > 1

Requir

Q. Software to predict diseases using X-Ray images with Computer Vision

1. Feasibility Study:

>entry: Problem given
(to predict diseases)

Technicality:

- Points needed are scanning the X-ray / option to upload the X-ray image.
- function to feed the model
- function to check sample cases.



2. Requirement Analysis & Specification

- End Users:

>Big / Large Hospitals, Pharmacies.

> Doctors.

- Type of data:

csv data scanned from X-Rays with specific columns.

- Form of data:

Class-18

Software Engineering

Dt. 08/08/2025

Classical Waterfall Model

3. Design → (A) Structured Approach

- (a) Structured Analysis → Analysing SRS → getting Data flows b/w modules/ submodules.
- (b) Structured Design.
  - (i) High level (Architectural) → expands the modules in all possible submodules and interface b/w them
  - (ii) Low level (Detailed) → Data Structures / functions
    - Data Structures
    - Organisation

exit criteria : Architectures, Data Structures Used, Algorithms of the functions /sub-functions / sub modules.

To convert the SRS to Architectural format with some detailed illustration.

↳ (B) Object Oriented Designing Approach

4) Coding & Unit Testing:

entry criteria: DFD, Flowcharts,
↳ Data Flow Diagrams

→ converting algo in codes.

exit criteria: coded and tested subfunctions.

5) Integration & System testing:

- Interlinking all the subfunctions and modules.
- System testing may be ~~"corrective"~~ →
  - α-Test → experts test this one
  - β-Test → known customers do this.
  - Acceptance Test.
    ↳ All customers.

exit criteria: Fully tested software/mode/system.

6) Maintainance:

- Corrective measures
- Performance based measures./perfective → by adding new features
- Adaptive measures.
  ↳ Improving the scalability.

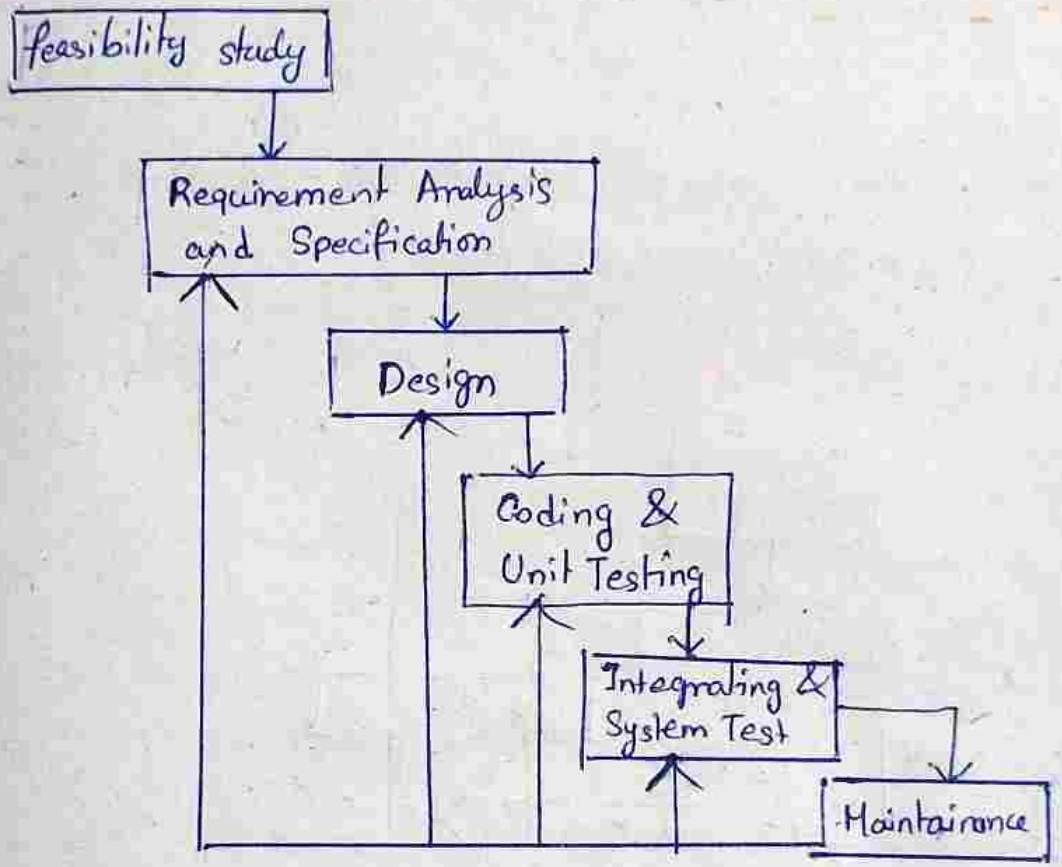
NOTE:
error
↳ execution of code.

Phase Containment Error:

- To remove the future risk, this error has to be minimised

drawbacks: The chance of phase containment error is →
- phase overlap not possible. → rigid implementation of phases one after another
- no feedback from one phase to another to trash/debug the problem. → more time needed to develop the software
  → most of the team will be idle
  → Wastage of (human) resources.

Iterative Waterfall Model :



- Once you do the feasibility study, you can go back to any required stage.

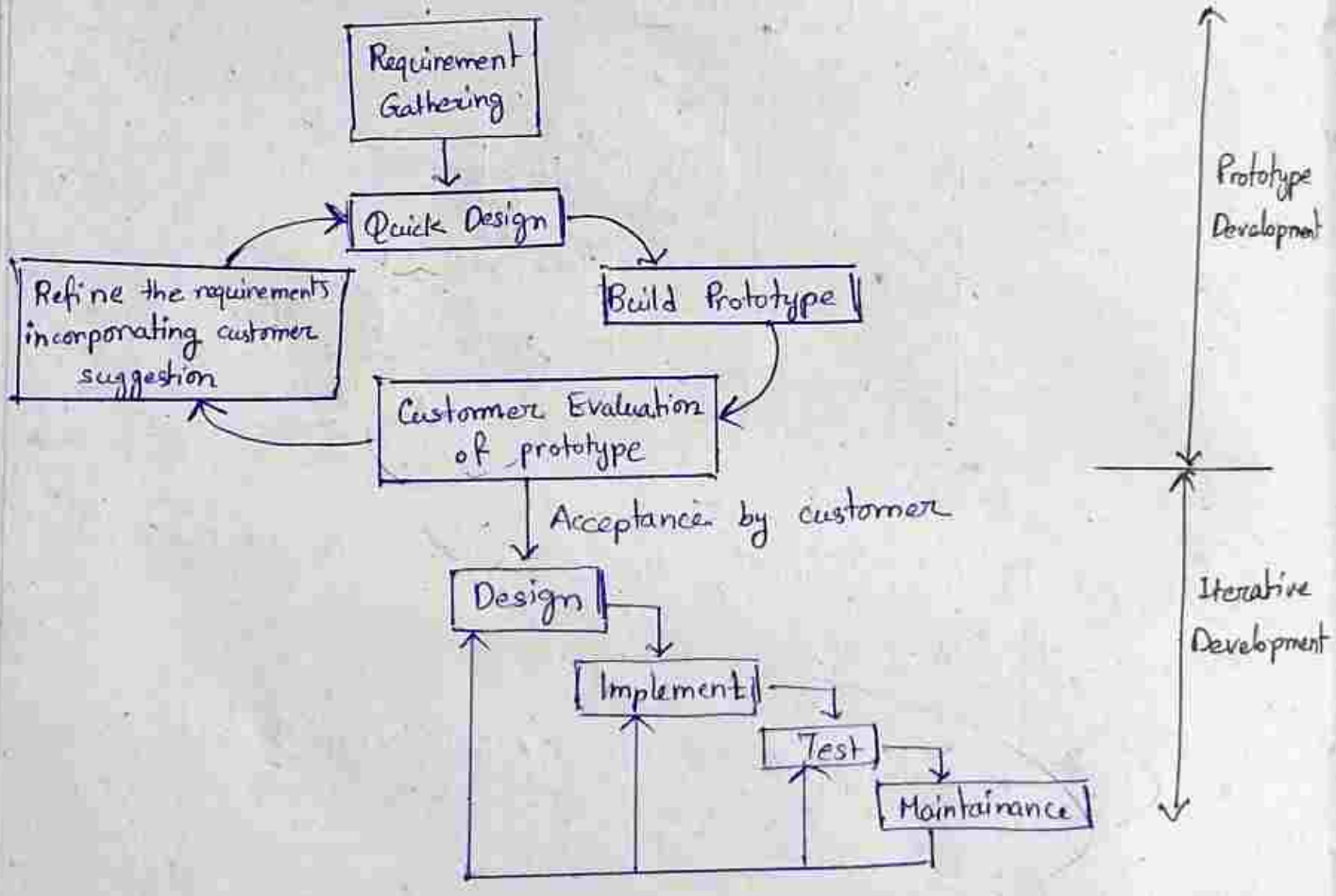
drawbacks

- rigid
- waste of resource
- Utilisation is less

Phase Overlap :

TO PROTOTYPING MODEL :

→ reliability limited.
→ more suitable for those whose risks are not fully identified
→ more suitable for projects, more GUI based.



- It is an extension of classical waterfall model.
- It suggests to build a ~~walking~~ working prototype of the system, before developing the actual software.
- The prototype is the toy and crude implementation of a system with limited functional capabilities, low reliability or inefficient performance.
- It is most suitable to projects
  - containing GUI parts of an application
  - exact technical solutions are unclear.

> Strengths:

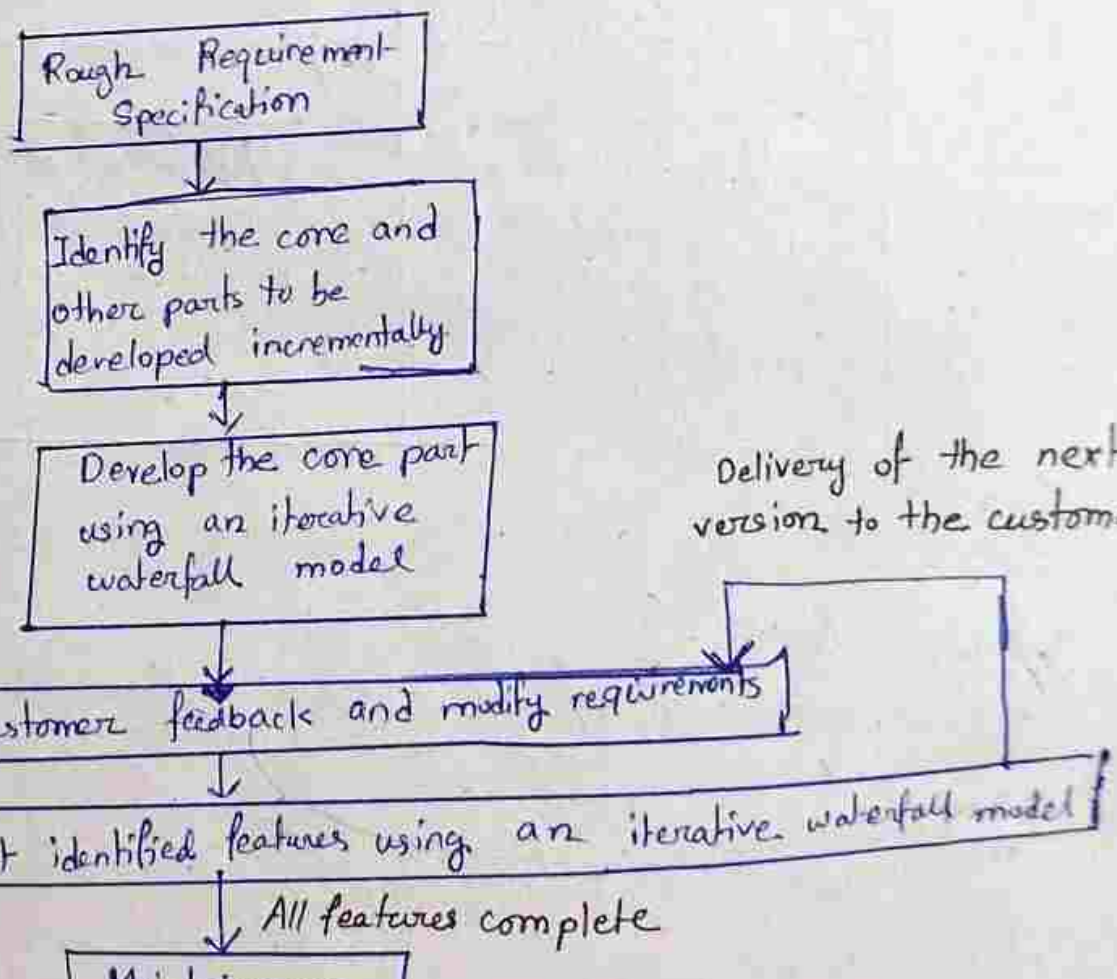
- Appropriate for the projects that suffer from risks arising from technical uncertainities and unclear requirements.

> Weaknesses:

- It increases the cost of development.
- It is only effective for those projects for which the risk can be identified upfront before the development starts and it is ineffective for risks identified later.

EVOLUTIONARY MODEL

- ~~Idelytofi~~
- Sometimes, it is referred to as "Design a little, Build a little, Test a little, Deploy a little" model.
- It means after the requirement has been satisfied, the design, build, test and deployment activities are iterated.



> Strengths:

- It fulfills all the actual customer requirements.
- Easy handling change requests.

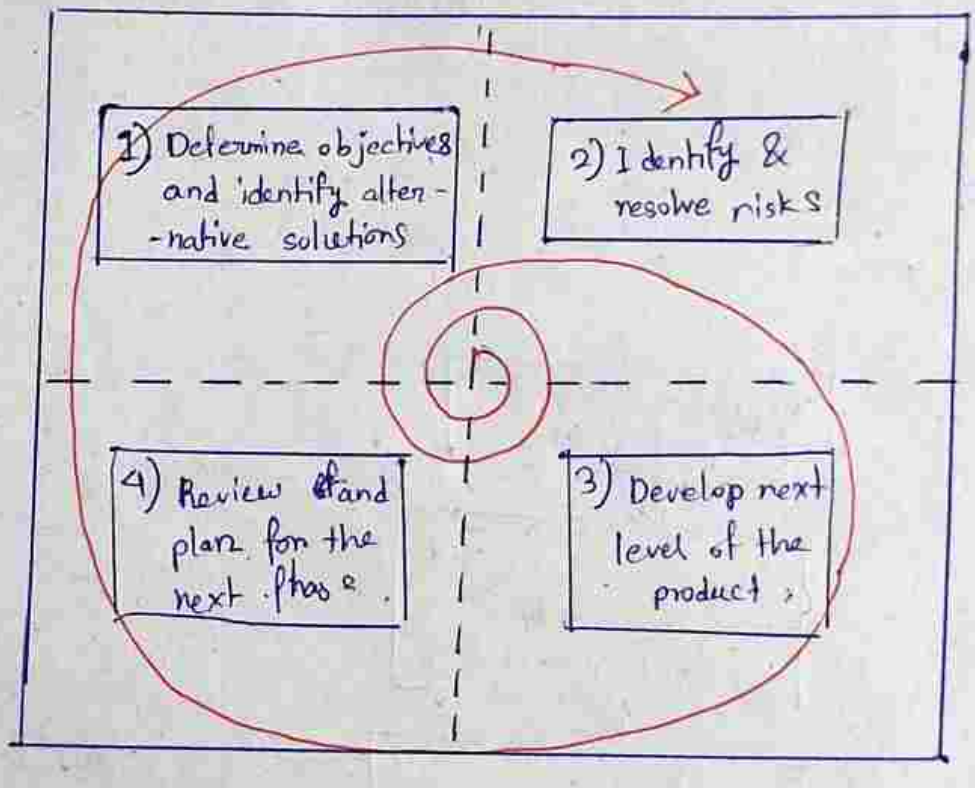
> Drawbacks:

- Feature division into incremental parts can be non-trivial.
- It is an adhoc design.

SPIRAL MODEL

~~Many loops, which can~~

- It contains many loops, which can vary project to project.
- Each loop of the spiral is called a phase of the software model.



> Drawbacks

- It is more complex in nature. → time consuming.

> Strengths

- Higher risk management.

Q. 1 Compare and contrast the following SDLC models.

a) Classical waterfall.
b) Iterative waterfall.
c) prototyping model.
d) evolutionary model.
e) spiral model.

Q. 2 If the prototyping model is being used by a team developing a moderate sized software, is it necessary to develop an SRS document? justify your answers.

Q. 3 Explain how a Software Development Effort is initiated and finally terminated in the Spinal model.

Q. 4 While using the iterative waterfall model to develop a commercialised software for a business application, discuss how the effort spent on the different phases is spread over time.

Q. 5 What do you understand by the 99% complete syndrome that the Software managers sometime face?

Q1)

Class-26 Software Engineering Dt. 18/08/25

Software Project Management :

- Job of SW Project Manager → (I) Coordinating between the teams
  (II) Keeping track of the progress (Monitoring & Control.)
  (III) Project Planning

- Size estimation → (a) Cost (line of codes)
  (b) time

- The main goal of software project management is to enable a group of developers to work effectively towards the successful completion of a project.

Software management Complexities.

- invisibility: (of progress of particular stage.)

- changeability :

- uniqueness :
  Checking if the project is unique or not. (is not already made before.)
  (bypassing copywrites.)

- exactness of solution

- Team oriented and intellect oriented work:

- If the SWD process

Responsibility of a Software Manager

(a) Project planning → done by SWM after the feasibility study and before Requirement Analysis

(b) Project Monitoring & control.

↳ It's undertaken once the development activities start.

Skills of Software Project Manager

1. Knowledge (about the project management techniques.)
2. Decision-Making capabilities.
3. Previous experience (in the similar types of projects.)

Project Planning

(i) estimation → cost, time, effort

(ii) Scheduling → sequence/order of implementation/usage of manpower and other resources.

(iii) Staffing → Staff organisation & staffing plans.

(iv) Risk Management → (risk identification + analysis + elimination)

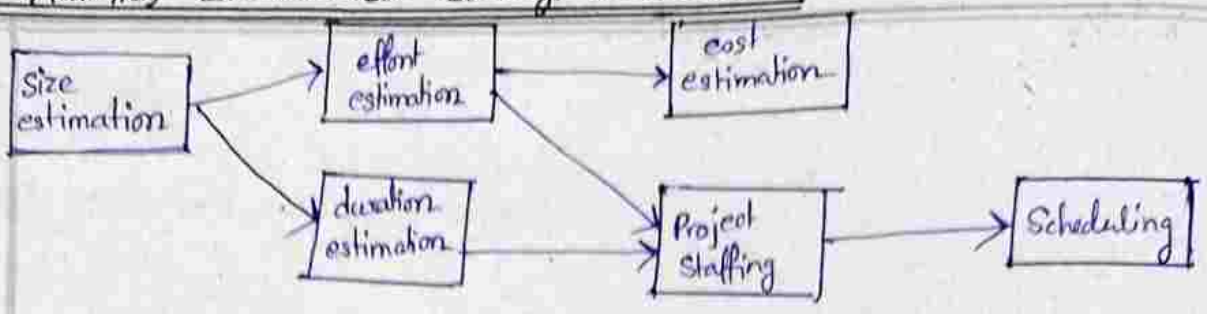
(v) Miscellaneous plans → it ensures you quality assurance

- Size is the most fundamental parameter based on which other estimations and project plans are made. (diagram in next page.)

- **Sliding Window Planning:**
  - In this planning technique, starting with an initial plan, the project is planned more accurately over a number of stages.

- **The SPMP document:**
  (Software Project Management Plan.)
  - The project managers document their plans in a SPMP document.

Activities carried out during estimation



SPMP document

① Introduction:
- objectives
- Major functions
- performance issues
- Management & Technical constraints.

② Project estimates:
- Historical Data Used.
- Estimation Techniques used.
- Effort, Resume, Cost, & Project Duration estimates.

③ Schedule:
- Work breakdown structure.
- Task network Representation.
- Gantt Chart Representation.
- PERT Chart Representation.

④ Project Resources:
- People
- Hardware and Software.
- Special Resources.

⑤ Staff Organisation:
- Team Structure.
- Management Reporting

⑥ Risk Management Plan:
- Risk Identification
- Risk Analysis
- Risk Estimation
- Risk abatement procedures.

⑦ Project Tracking & control plan:
- Metrics used to tracked.
- Tracking plan & control plan

⑧ ~~Miscellaneous~~

⑧ Miscellaneous:
- Process Tailoring
- Quality Assurance Plan
- Configuration Management Plan
- Validation & Verification
- System Testing plan
- delivery, installation & maintainance plan.

• Metrics used for size estimations

two metrics are used.

① LOC (Lines of Code) → Total no. of executable instruction.
② FP (Function Point) → It's the simplest metric used to ~~measure~~ measure project size by counting the no. of source instruction in the ~~develope~~ developed problem.
→ The comment & header lines are ignored.
→ ~~Determi~~ Determining LOC can't at the end of the project is very simple but very difficult at the beginning.
→ However, using some systematic guess work one can possibly estimate the LOC count.
→ ~~The sy~~

The systematic guesswork involves —

① The project manager ~~devices the~~ devides the problems into modules and each modules into submodules until the line of codes of leaf level modules are small enough to be predicted.
② The total size of the project can be estimated by ~~any~~ adding the estimations of all leaf level modules.

#Note:
- Past experience in previous projects is highly beneficial/essentia

H/W

Q. Illustrate the shortcomings of LOC.

(2) Function Point Metrics

- Try to figure out key function and what kind of output it provides.
- what kind of files are used.

→ function → Input → output → files need to be accessed → interface.

Class-35

Page No.:
Date: 22/08/25

Software Engineering

Function Point Estimation:

- Instead of dividing the problem, it is better to see the problem as a collection of function.
- Each function will have a certain input/output.
- Functions — Input, Output — Files/DB, Interface, complexities.

> Conceptually, the size of the software product is directly dependant on the number of different high level functions or features it supports.

> The computation of number of inputs, outputs, files, and interfaces would give a more accurate indication about the size of the software.

Interface: It refers to the different mechanisms by which the user transfers the data with external systems.

Step I: Compute Unadjusted function points (UFP).

$$UFP = (\text{no. of inputs}) \times 4 + (\text{no. of outputs}) \times 5 + (\text{no. of files}) \times 10 + (\text{no. of enquiries}) \times 4 + (\text{no. of interfaces}) \times 10$$

- When information are related then it is considered as one..(for input)
- Content related to the same kind of enquiry is treated as one.
- Sir Albrecht proposed this method.

Step II: Refinement of the UFP

3 levels of complexities are considered → simple, average, complex
↳ (of the parameters of UFP)

TYPE	SIMPLE	AVG	COMPLEX
I	3	4	6
O	4	5	7
E	3	4	6
F	7	10	15
N	5	7	10

Step III: Compute FP (Here, we find degree of influence (DI))

- Albrecht et al. identified 14 parameters that can influence the development effort and these are —
  1) reliable backup and recovery
  2) data communication
  3) distributed processing
  4) performance requirements
  5) operational environment
  6) online data entries
  7) multiscreen/multioperation data input
  8) online updating of masterfiles
  9) complex input/output/queries/files
  10) complex data processing
  11) code to be designed for reuse
  12) conversion and installation
  13) multiple installation in an organisation change
  14) focus on ease of use.

- each of these ranges from 0 to 6.
  (0 → no influence, 6 → strong influence)

DI

- We can compute the degree of influence which ranges from 0 to 89.
- TCF (Technical Complexity Factor) = $(0.65 + 0.01 \times DI)$

$$\boxed{FP = TCF \times UFP}$$

Q. Complexity ~~of~~ waiting factors for I, O, E, F, N are 4, 5, 4, 10, 7 respectively. It is given that out of 14 values adjustment factors, 4 factors are not applicable, each of the other 4 factors have the value 3 and the remaining has the value 4, note that the no. of I, O, E, F, N are 30, 60, 23, 8, 2 respectively. Compute FP.

Ans. $DI = (4 \times 3) + (6 \times 4) = 12 + 24 = 36$

$UFP = (4 \times 30) + (5 \times 60) + (4 \times 23) + (10 \times 8) + (2 \times 7)$

$= 606$

$TCF = (0.65 + 0.01 \times 36) = 0.65 + 0.36 = 1.01$

$\therefore$ Ans $\Rightarrow FP = UFP \times TCP = 606 \times 1.01$

$= 612.06.$

Class-38

Software Engineering

Dt. 28/8/25

• Project Estimation Techniques

▲ Project Estimation → Project size
→ Duration
→ ~~Or~~ Cost
→ Effort

▲ Project Estimation Techniques → Empirical → Expert judgement
→ Delphi cost estimation
→ Heuristic
→ Analytical

• Heuristic :

↳ COCOMO (COnstructive COst estimation MOdel) :

- Cost depends directly on duration and effort.
- COCOMO gives a systematic way to find duration and effort.
- It follows 3 stages. → (a) Basic COCOMO
(b) Intermediate COCOMO
(c) Complete COCOMO

• Basic COCOMO → Initial estimation is done.
(The factors affecting the project development is not considered)

Page No.:
Date: / /

• Intermediate → Identify the affecting factors, and then update the estimation coming from Basic COCOMO.

② Basic COCOMO:

• At first, in which class the SW lies on falls before estimating.

• SW classes
  - → Organic → (more application oriented)
  - → Semi detached → (utility based softwares)
  - → Embedded → (system softwares)
    - → OS | RTOS | DOS etc.
    - → compilers
    - → linkers | loaders

• Estimation utilise these 2 co-efficients:
  - KLOC (Kilo Lines of Codes) → directly proportional to project size
  - PM (Person per Month) → indicates the efforts

(A whole program can be written in a single line, it will be counted as a single line.)

• The equations for effort and duration are:

$$\text{Effort} = a_1 \times (\text{KLOC})^{a_2} \quad \text{PM}$$

$$T_{dev} = b_1 \times (\text{Effort})^{b_2} \quad \text{Months}$$

(Months → unit)

• For organic:

$$\text{Effort} = 2.9\,(\text{KLOC})^{1.05}$$

$$T_{dev} = 2.5\,(\text{Effort})^{0.38}$$

• Semi detached :

$$\text{Effort} = 3.0 (\text{KLOC})^{1.12}$$
$$T_{dev} = 2.5 (\text{Effort})^{0.35}$$

• Embedded :

$$\text{Effort} = 3.6 (\text{KLOC})^{1.20}$$
$$T_{dev} = 2.5 (\text{Effort})^{0.32}$$

Q. Assume that the size of an organic type software product has been estimated to be 32000 lines of source codes. Assume that the avg. salary of a software developer is 15000/- per month. Determine the effort required to develop the software product and the development time and the staff cost required to develop the product.

Ans. KLOC = 32

$$\text{Effort} = 2.9 (\text{KLOC})^{1.05}$$
$$= 2.9 (32)^{1.05}$$
$$= 2.9 \times 38.05 = 91.33 \text{ PM} \approx 91 \text{ PM}$$

$$T_{dev} = 2.5 (\text{Effort})^{0.38}$$
$$= 2.5 \times (91.33)^{0.38}$$
$$= 2.5 \times 5.56$$
$$= 13.89 \text{ Months} \approx 14 \text{ Month}.$$

∴ Staff cost = 91.33 × 15000 = 1365000

per month = 1369966.69.

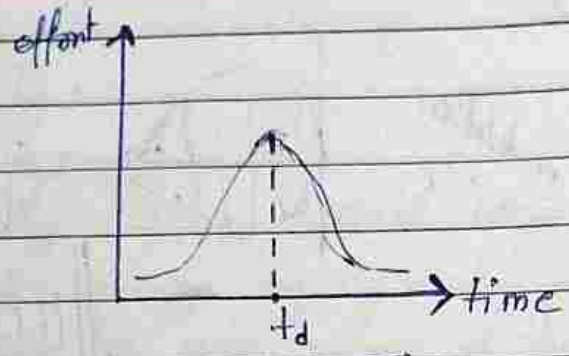
∴ Staff cost total = 1369966.69 × 14/- (1365000 × 14)/-

= 19179532.36 = 19110000/-

class 42

Staffing level Estimation:

» Norden's Work:



Norden showed staffing pattern in R&D can be approximated by Rayleigh Distribution

$$E = \frac{K}{(t_d^2)} \times t\, e^{-t^2/2t_d^2}$$

K = area under curve currently.
t_d = time when effort is max
t = current time

» Putnam's Work:

$$L = C_k K^{1/3} t_d^{4/3}$$

Product size in KLOC

K = total effort in PM
C_k = state of technology constant
t_d = time taken in integration & testing

$C_k = 2$, for poor development environment
$= 8$, for good development environment
$= 11$, for excellent development environment.

$$K = \frac{L^3}{(C_k^3 t_d^4)} \quad \text{or} \quad K = \frac{c}{t_d^4} \quad \left(c = \frac{L^3}{C_k^3} = \text{small constant.}\right)$$

$$\Rightarrow \boxed{\frac{K_1}{K_2} = \left(\frac{t_{d_2}}{t_{d_1}}\right)^4}$$ with this we see how when schedule is compressed the effort increases by power of 4.

» Jensen's Model

$$\boxed{L = C_{te}\, t_d\, K^{1/2}}$$

C_{te} = effective technology constant
K = Effort needed to develop the software
t_d = Time to develop software

$$\Rightarrow \boxed{\frac{K_1}{K_2} = \frac{t_{d_2}^2}{t_{d_1}^2}}$$ Here the increase in effort is proportional to square of degree of compression

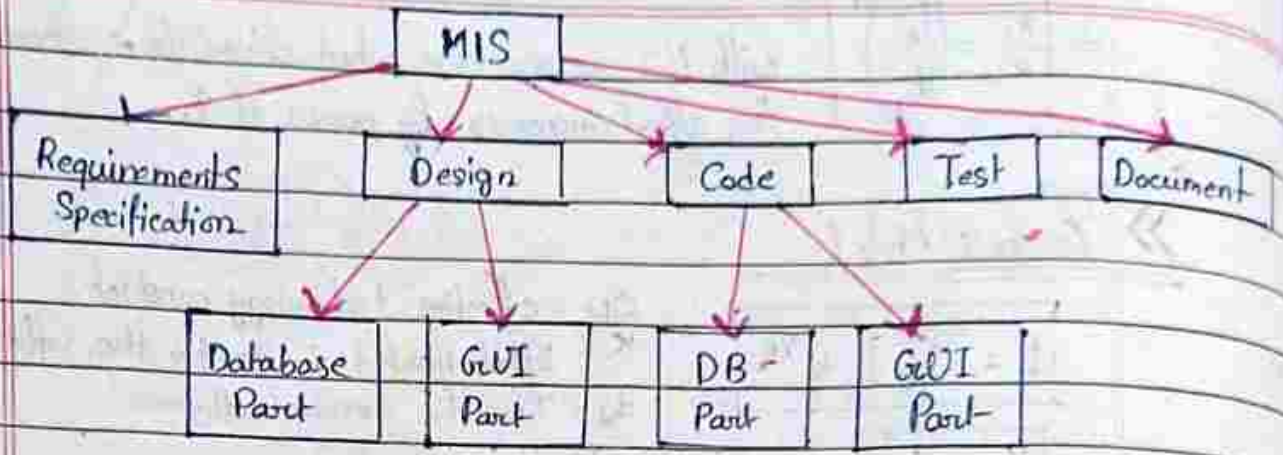
2 How Effort and Project Size Influences Sofware Development Project

The scheduling problem consisting of which task could be taken up when and by whom, for this the software manager need to do following things.

1. Identify the major activities to complete the project
2. Break down each activity into tasks.
3. Determine the dependancy among different task.
4. Establish the estimate for the time duration necessary to complete task
5. Represent the information in form of an activity network.
6. Determine start and end of the task.
7. Determine critical path.
8. Allocate resources to task.

» SCHEDULING

(1) Work Breakdown Structure: Here we will decompose a given set of activities into smaller activities recursively. Each of the activity is represented by a rectangle.



② Activity Networks

- It's represented in two ways — (i) AoN (ii) AoE
- It shows different activity making up a project and their internal dependancy.

• AoN (Activity on Node)

Activity represented on rectangle or circle including duration of each task.

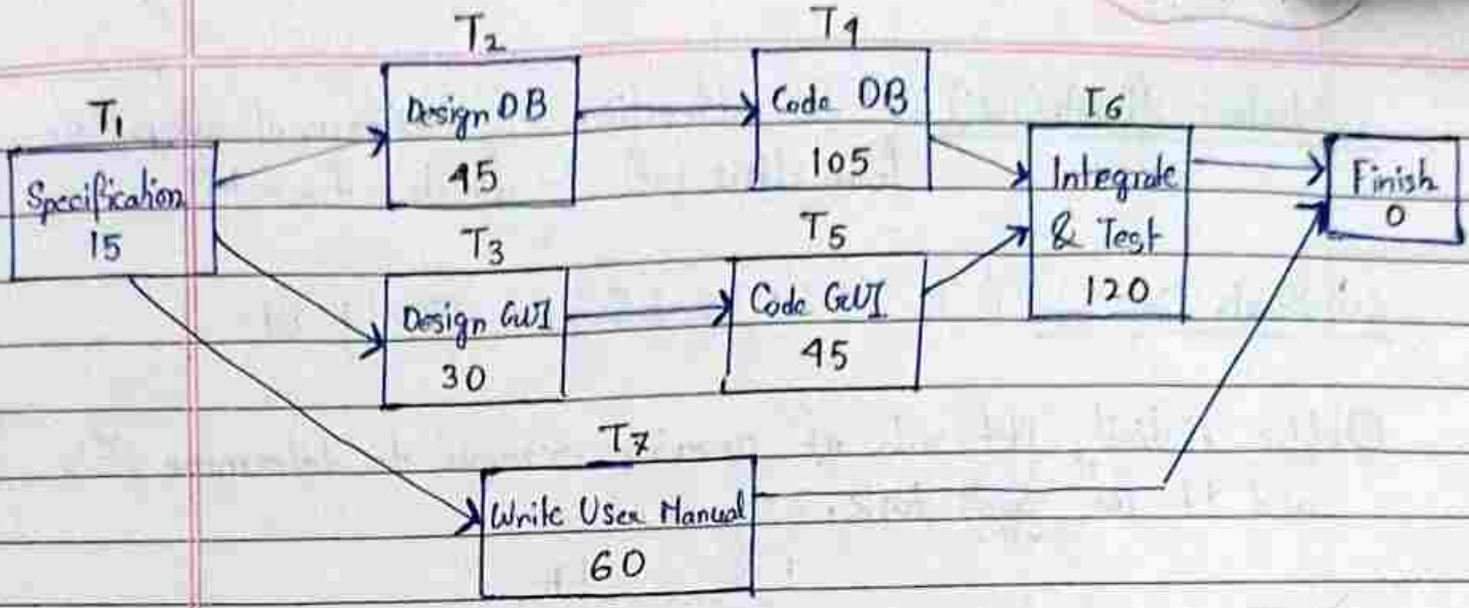
- The inter-task dependancy can be represented by edges.

• AoE (Activity on Edges)

Activity tasks are associated represented with the edges and edges are associated with duration. Nodes represents project milestone.

Q. Determine the activity network representation for MIS development project where relevant data is given below.

No.	Task	Duration	Dependant on Task
T_1	Specification	15	-
T_2	Design DB	45	T_1
T_3	Design GUI	30	T_1
T_4	Code DB	105	T_2
T_5	Code GUI	95	T_3
T_6	Integrate & Test	120	T_4, T_5
T_7	Write User Manual	60	T_1



③ CRITICAL PATH METHOD:

- Slack Time is the time that a particular task can be delayed such that total time is not affected.
- Critical Task is a task where slack time is zero.
- Critical Path is the sequence that leads through critical task. OR, a path which starts from starting node to the finish node only containing critical tasks.

Following quantities need to to calculated:

(i) Minimum Time: (Min time to complete the project.) Determined by taking maximum of all paths from start to finish.

(ii) Earliest Start: ES of a task is ES of previous task plus the duration of preceding task.

(iii) Latest Start Time: Difference b/w MT and the maximum of all paths from this task to finish.

(iv) Earliest Finish Time: EF is the sum of ES of a task and the duration of the task.

(v) Latest finish (LF): LF is subtraction of maximum of all paths from this task to finish from MT.

(vi) Slacks Time: it is ST = LS - ES or ST = LF - EF

Q. Use Activity Network of previous example to determine ES and EF for each task.

		ES	EF
⇒ T_1	Specification :	0	15
T_2	Design DB :	15	60
T_3	Design GUI :	15	45
T_4	Code DB :	60	165
T_5	Code GUI :	45	90
T_6	Integrate :	165	285
T_7	Write User Manual :	15	75

Class 49

Software Engineering

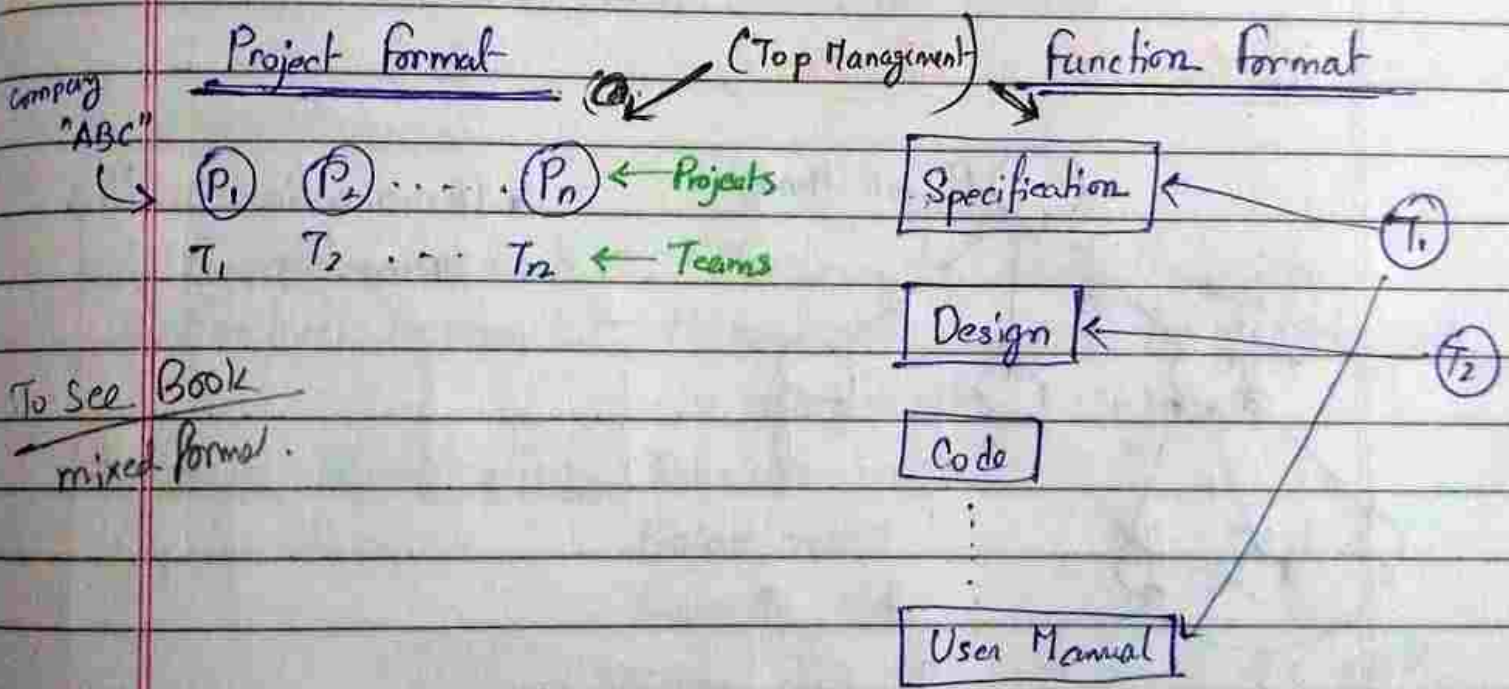
Dt. 09/09/25

④ Gantt Chart



ORGANISATION & TEAM STRUCTURE

Organisation Structure:

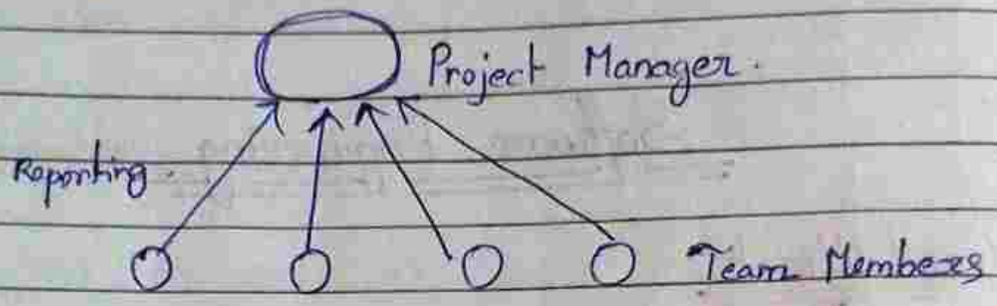


To see Book mixed format.

Page No.:
Date: / /

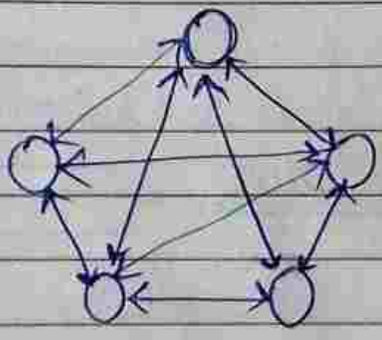
Team Structure:

(i) Chief Programmer Team Structure:



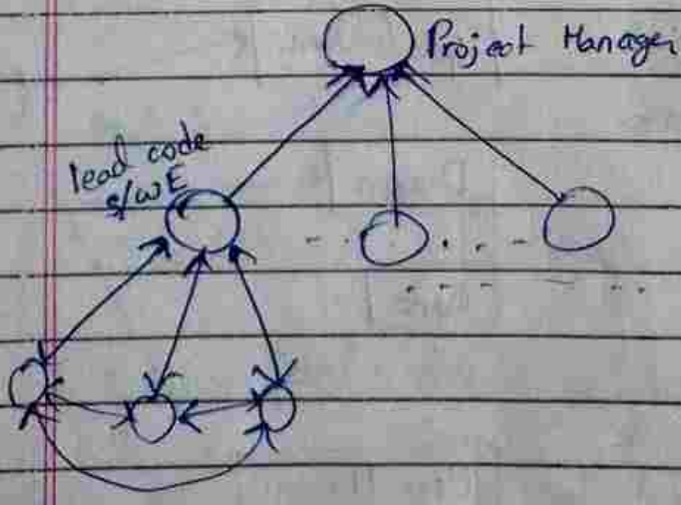
(ii) Democratic Team:

A mess kind of structure, anyone can communicate with anyone.



- If members of Design or Coding teams take this structures, it might give better results.

(iii) Mixed Team Structure:



- Democratic in the branches.

STAFFING :

(i) ~~Good communication skills~~

Selecting the software developers is too crucial for the success of a project. A good software engineer should have following qualities :-

(i) Good technical knowledge about the project areas.
(ii) Good programming skills
(iii) Good communication skills
(iv) Sound knowledge of fundamentals of computer science.
(v) Intelligence.
(vi) Ability to work in a team.
(vii) Discipline.

Risk Management :

- Risk is unfavourable circumstances which can be triggered during the development of the project.

Approaches :

(i) Proactive Approach → Measures to avoid the risk
(ii) Reactive Approach → When risk happens take measures to remove it.

→ **Risk Identification :** Technical risks
Project risks.
Business risks

→ **Risk Assessment :**

$$P = r \times s$$

P = priority of the risk.
r = likelihood of a risk becoming real
s = consequence of the problems associated with the risk.

→ Risk Mitigation:
- → Reduction
- → Avoid
- → Transfer

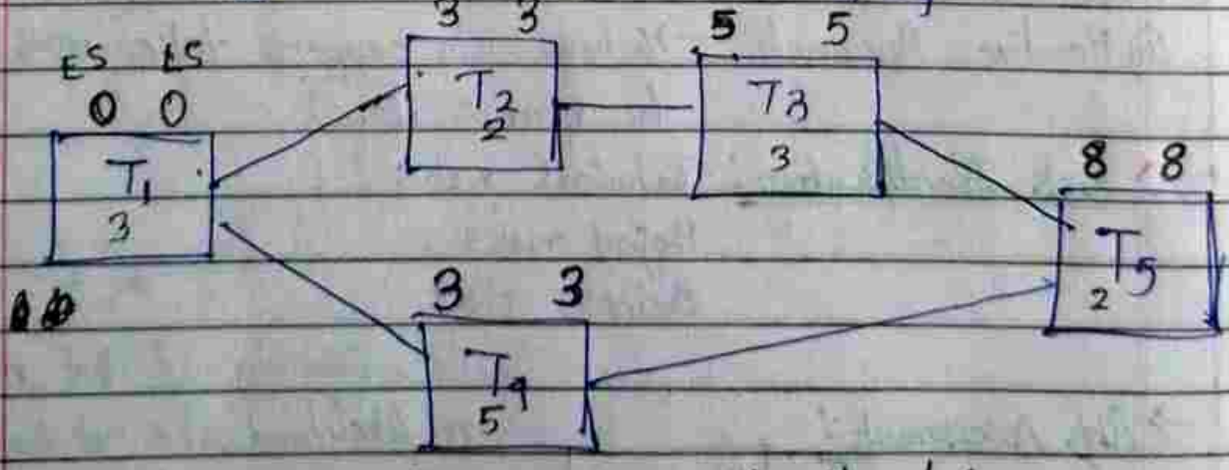
Configuration Management:

Controlling and managing the S/W project deliverables.

Q. Consider a software project with 5 tasks $T_1, \ldots T_5$. Duration of the 5 task in which are 3, 2, 3, 5, 2 respectively. T_2 and T_4 can start when T_1 is complete. T_3 can start when T_2 is complete T_5 can start when T_3 and T_4 are complete. Draw the AoN. ~~When is the latest~~ What is the slack time of task T_4? What tasks are on the critical path?

Ans:

Tasks	Time	Depends
T_1	3	-
T_2	2	T_1
T_3	3	T_2
T_4	5	T_1
T_5	2	T_3, T_4

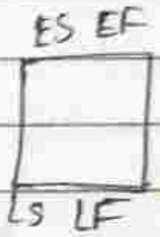


∴ Slack Time = 0 of T_4

∴ All the tasks are in critical path

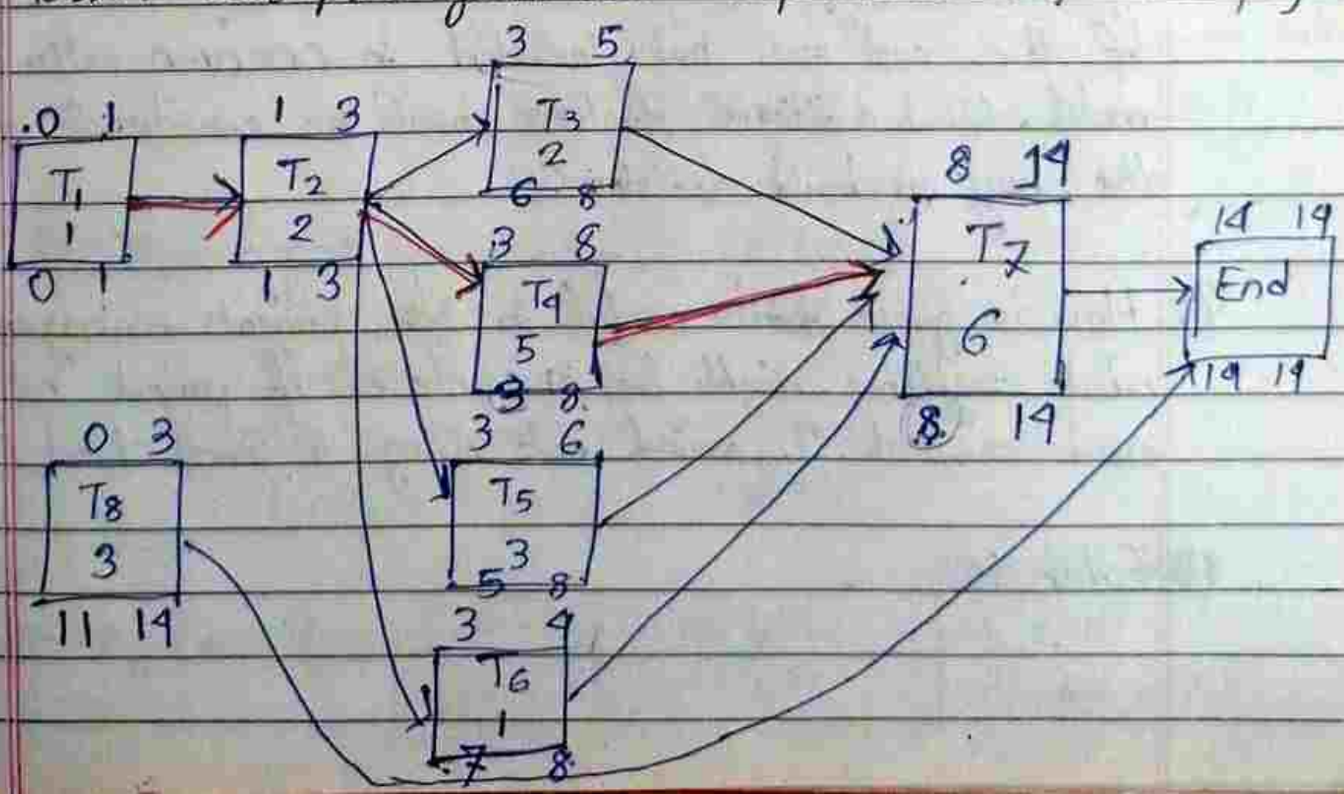
Q. The following table indicates various tasks involved in completing a s/w project, the corresponding activities, the estimated effort for each task in PM.

Notation	Activity	effort in PM
T_1	Requirement Specification	1
T_2	Design	2
T_3	Code actuator interface module	2
T_4	Code sensor interface module	5
T_5	Code user interface part	3
T_6	Code control processing part	1
T_7	Integrate & Test	6
T_8	Write user Manual.	3



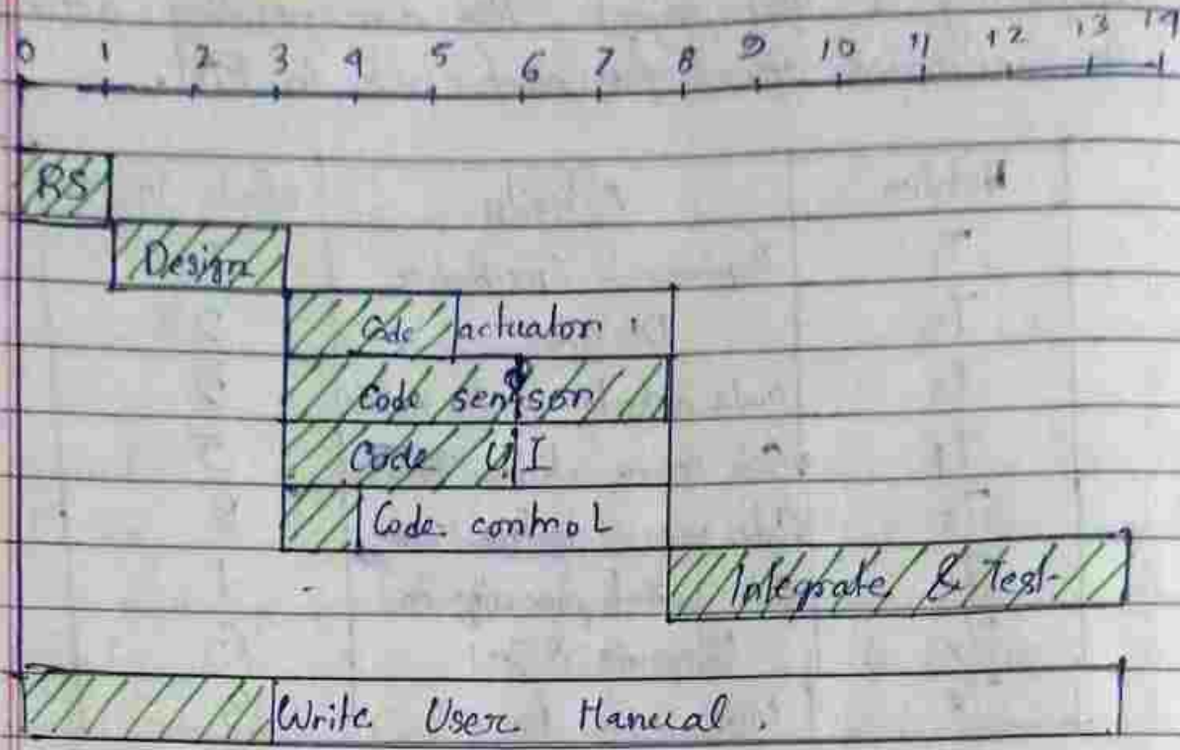
The precedence relation $T_i \leq \{T_j, T_k\}$ implies that the task T_i must complete before T_j and T_k can start. The following precedence relation is known to hold among different tasks. $T_1 \leq T_2 \leq \{T_3, T_4, T_5, T_6\} \leq T_7$. Draw the activity network, determine ES, EF, LS, LF for every task. Develop the gantt chart representation for the project.

Ans



Page No.
Date : / /

Gantt Chart :



Q. A certain s/w product for business application costs 50L to buy off the shelf and its size is 40 KLOC. Assuming that in house interns cost 60000 per programmer month (including overheads), ~~put if~~ could it be more cost effective to buy the product or build it? which elements of the cost are not included in COCOMO estimation model. What additional factors should be considered in making the buy or build decision?

Q. How is gantt chart useful in s/w project management? what problems might be encountered if project monitoring and control is carried out using a Gantt chart?

Q. ~~What is~~

Class - 57 (UNIT II) Software Engineering Dt. 13/9/25

Requirement Analysis & Specification :

Goal : → Understanding the requirements (of customers on the software.)
→ Systematic Organization (SRS).

Validate SRS : → by customers
→ Internally by project team
Ensures → Understandable
→ legitimate / unambiguous.
→ complete

Stages to obtain SRS

(i) Requirement Gathering
(ii) Requirement Analysis
(iii) Requirement Specification

Task → Service provided by the SW.

Date. _____

Page No _____

① Requirement Gathering:

→ Gather requirements from stakeholders.

↳ source of requirements

(person with directly involved/ indirectly involved with SW)

Q. When will be gathering of requirements very easy?

Ans. Already when the problem statement is used, i.e. people already tried to solve it.

- REQUIREMENT GATHERING will be done System Analyst (SA) when no working model of the SW is available.

- Ways to gather requirements:

Read from Book

→ Understanding the existing documents.
→ Interviews → Categories of users.
↳ Delphi approach.
→ Task Analysis.
→ Scenario Analysis
→ Form Analysis.

② Requirement Analysis :

To conduct the requirement analysis, the system analyst must clearly understand the answers of the following questions.

1. What is the problem?
2. What is its importance?
3. What exactly data input/output?
4. What are the procedure to solve the problem?
5. What are the complexities that may arise?
6. What should be data interchange formats with external system?

- SA needs to resolve → Anomaly
  → Inconsistency
  → Incompleteness

- Anomaly → Requirements can be interpreted in several ways.
  - ex: When the temp. becomes high, the heater should be switched off.
    sol: giving a threshold / quantifying (if temp > 100°C)

- Inconsistency → two requirements contradicting each other.
  - ex: R1: The furnace should be switched off when its temp. rises above 500°C.

Date.
Page No.

(R2) When the furnace temp. rises above 500°C, The water shower should be switched on and the furnace should remain on.

Sol: replacing one of these.

- Incompleteness:
  Some requirements are over looked.

Class-58 | Software Engineering | Dt. 16/9/25

(3) Software Requirement Specification:

- Documenting the requirement in a ordered format.
- SRS is used throughout the development stages.

(1) Users of SRS:

(A) Normal users / Customers / Marketing Personals
(B) Software developers
(C) Test Engineers
(D) User Document Writer.
(E) Project Manager.
(F) Maintainance Engineer.

(2) Why to spend time in developing SRS?

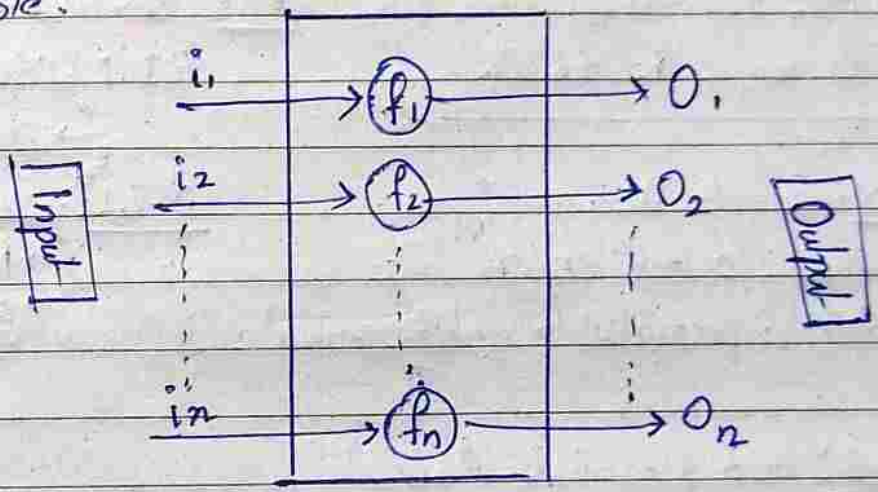
→ It acts as Agreement between customer and developer.
→ It reduces future rework.

Date
Page No.

→ It is used in estimating cost and duration/scheduling.
→ It is the basis for estimating & verification and validation.

3. Characteristics of a Good SRS document:

1. Concise and Informative.
2. The specified requirements should be implementation independant.
3. Trashable and modifiable.
4. Verifiable.



- The SRS document is also called as the Black Box specification of the SW being developed.

4. Attributes of a bad SRS:

1. Over Specification.
2. Forward Reference.
3. Wishful thinking
4. Noise.

⑤ Categories of Requirements :

According to IEEE 830 we have

- → Functional requirements
- → Non functional requirements.

• Functional requirements

F : I ⟶ O
f maps i_i to O_i

ex. service → "withdraw cash"

R1 : Withdraw cash
↳ Description

R-1.1 : Select Withdraw Amount
↳ Input : "Amount"
→ Output : User prompted to enter a/c type

R-1.2 : Select Account Type
↳ Input : Select SB/CA
→ Output : prompted to enter amount.

R-1.3 : Get Required Amount.
↳ Input : enter amount to be withdrawn
→ Output : ① prompted to enter pin
② for successful check ATM will provide money.

• Non functional Requirements

- > Non negotiable obligations must be supported.
- > Design and Implementation constraints
- > External Interfaces Required.
- > Other non functional requirements.

Date
Page No.

⑥ Organisation of SRS

1. Introduction
   - → purpose
   - → project scope
2. Overall Description of organisation of SRS doc.
   - → product perspective
   - → product features
   - → user classes
   - → operating environments.
   - → design & implementation constraints
   - → user documentation.
3. Functional Requirements.
4. External Interface Requirements
   - → User Interface
   - → Hardware Interface
   - → Software "
   - → Communication "
5. Other non-functional requirements
   - → performance requirements
   - → safety "
   - → Security "

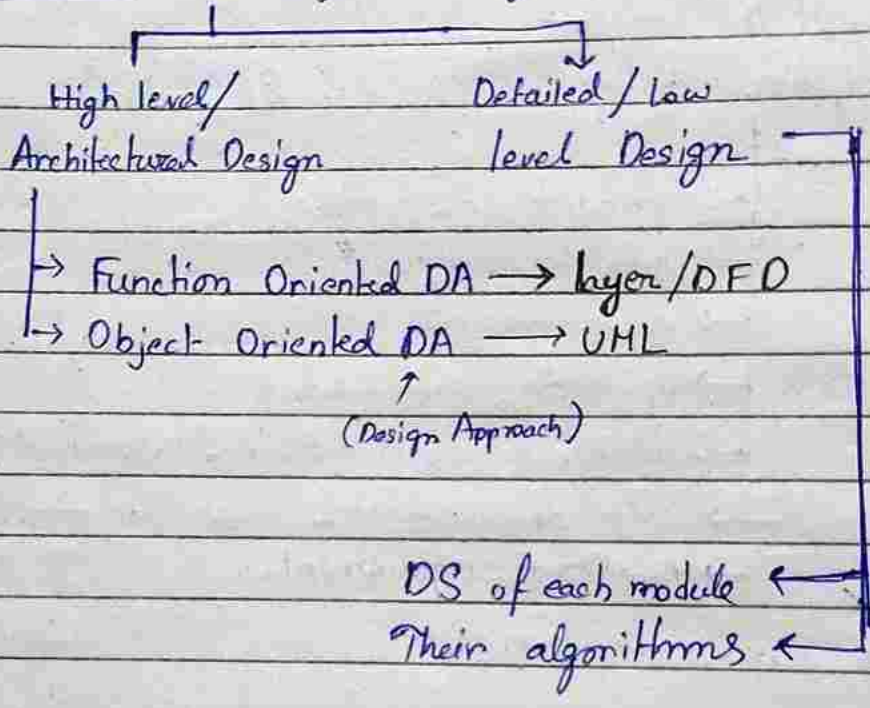
coupling → dependency among modules

cohesion →

Date
Page No.

SOFTWARE DESIGN

Input: SRS document
Output: SD (System Design)



Cohesion and Coupling:

Cohesion is a measure of functional strength of a module where as coupling between two modules is a measure of degree of interaction or interdependency between two modules.

- Good module decomposition is indicated through high cohesion of the individual modules and low coupling of the modules with each other.

Date
Page No.

Classification of cohesion

- Coincidental
- logical
- Temporal
- Procedural
- Communicational
- Sequential
- Functional

low ↓ high

Classification of coupling

- Data
- Stamp
- Control
- Common
- Content

low ↓ high

- Coincidental → Module performs set of related tasks.

class 63

Date 7/10/2025
Page No.

Software Engineering

Function Oriented Design:

- SRS → DFD → Structure chart.
- DFD is graphical view of functional requirements.

- These techniques, view system as a blackbox that provides set of services to the users of the software. These services are also called high level function.
- Function Oriented design follows top down decomposition of highlevel function, which are carried out to obtain more detailed function, this is called high level design.
- The identified functions are mapped to modules and submodules and module structure is created, such approach is known as low-level design.

High level design is same as →

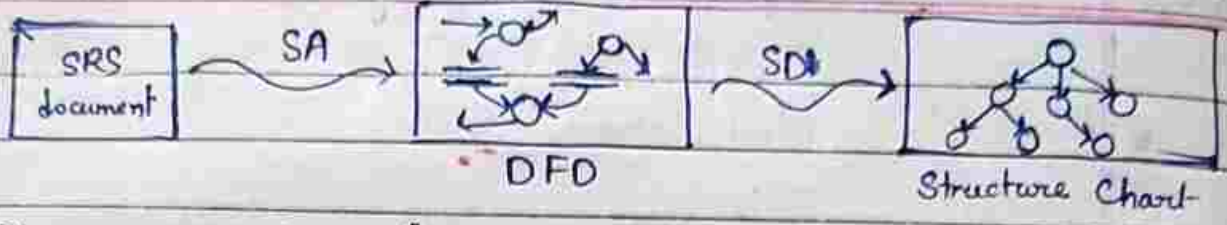
Structured Analysis (SA): SRS → DFD

Structured Design (SD): DFD → Structure Chart

Structured Analysis:

- The SRS document is transformed into a dataflow diagram model.
- The DFD model is transformed into structured chart.

Date.
Page No.



- The SA works on following principals —
  (i) Top down decomposition
  (ii) Application of Divide and Conquer principals
  (iii) Graphical representation of Analysis result using DFD.

DFD:

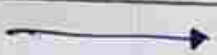
Hierarchical and graphical model of a system that shows different activities or functions that the system performs and the data interchange among those functions.

- It is also known as Bubble chart.

→ Primitive Symbols Used:

Process → 

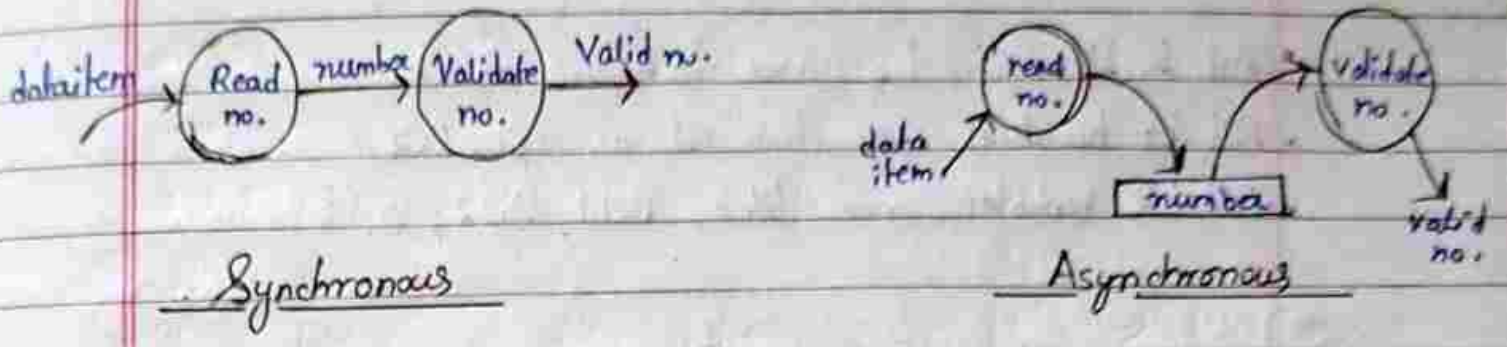
Data Store → 

Data flow → 

External Entity → 

↳ This interact with the system by inputting data through the system or by consuming data produced by the system.

Output :  when a hardcopy is produced.



Data dictionary :

It lists the purpose of all data items and the definition of all composite data items in terms of their component data items.

- Every DFD model of a system must be accompanied by a data dictionary

Book

- Composite data items can be done by combining primitives using, — "+", "[,]", "()", "{}", "=", "/* */".

- DFD can be upto :
  level 0: Context Diagram
  level 1: High level
  level 2: high level to sub-functions.

- L0: represents entire system as a single bubble; most abstract data flow representation of system.

- L1: The L1 DFD may contain upto 7 separate DFD. Generally it contains 3 to 7 bubbles representing a

Date
Page No

system performing 3 to 7 functions.

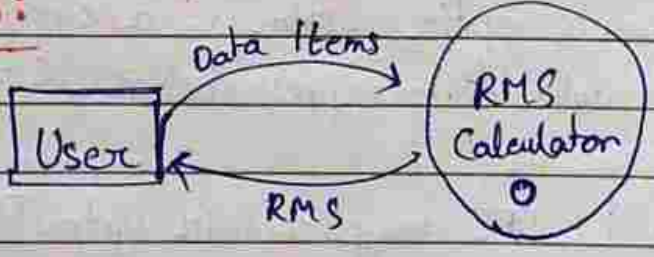
- All bubbles in L0 are labeled as 0.
- All L1 bubbles are labeled as 0.1, 0.2 ..... 0.3 likewise
- All L2 bubbles are like 0.11, 0.12, 0.13 ..... .

Q) RMS Calculating Software:

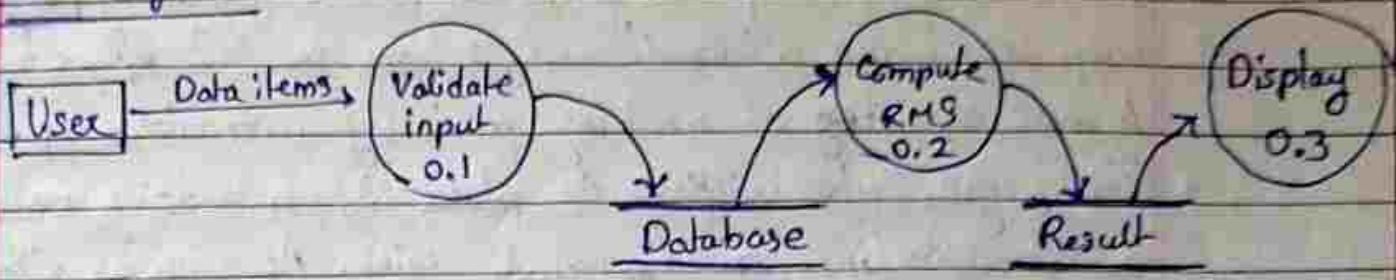
- A software system called RMS calculating software would read 3 integral numbers from the user in range of -1000 & +1000 and determine the RMS of them and display it.

- Functional Req:
  - 1.1. Validating:
    - Input: numbers
    - Output: validated numbers.
  - 1.2. Calculation:
    - Input: Validated numbers
    - Output: Calculated ~~number~~ ans
    - Process
      - 2.1. Sq. no1
      - 2.2. Sq. no2
      - 2.3 Sq. no3
      - 2.4. Mean
  - 1.3. Display:
    - Input: Calculated ans
    - Output: Display the ans.

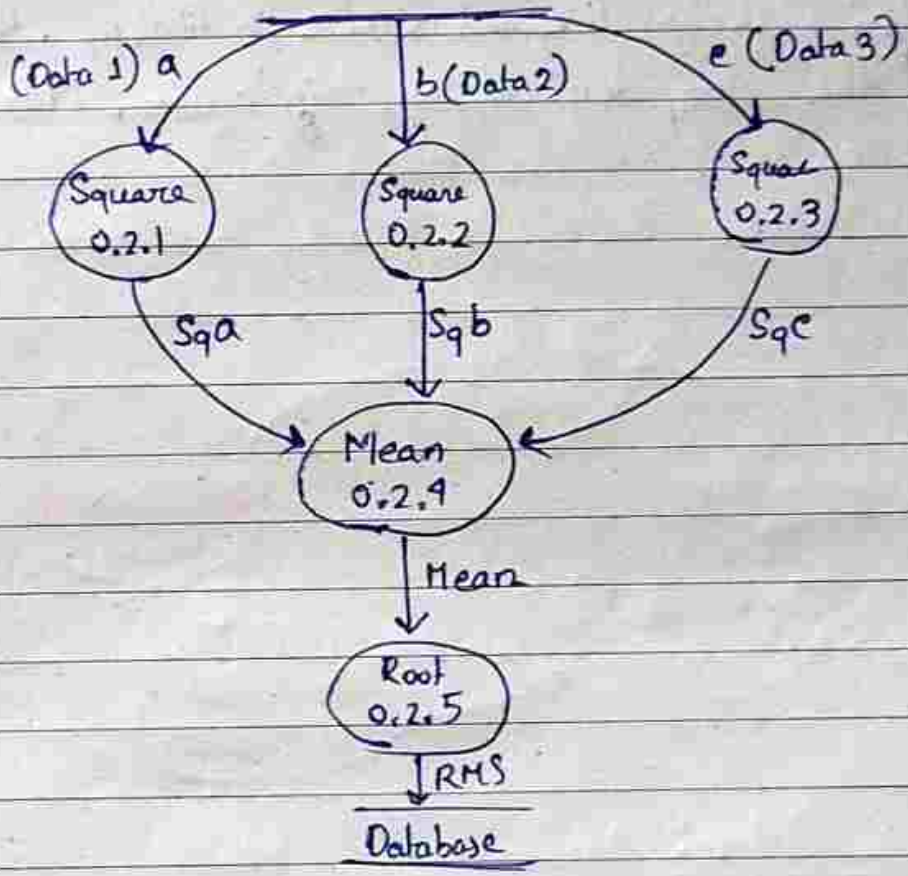
L0 Diagram:



L_1 Diagram:



L_2 Diagram:



Dic:

a, b, c : int, int, int

Mean : Double, float

RMS : Doub, float.

Sq : a*a | b*b | c*c

Mean : (a*a + b*b + c*c)/3

RMS : $(Mean)^{1/2}$

H/W

Date _____
Page No. _____

Q. A Tic Tac Toe computer game in which a human player and computer make alternate moves on a 3×3 square grid. A move consisting of marking previously unmarked square. The player who is first to place 3 consequtive mark along a straight line or diagonal wins. As soon as either human or computer wins; A message congratulating the winner is displayed; if neither manages to get 3 consequetive marks along a straight line and square is filled up, the game is drawn, and the computer always tries to win. Make DFD for this.

Ans. Functional Req:

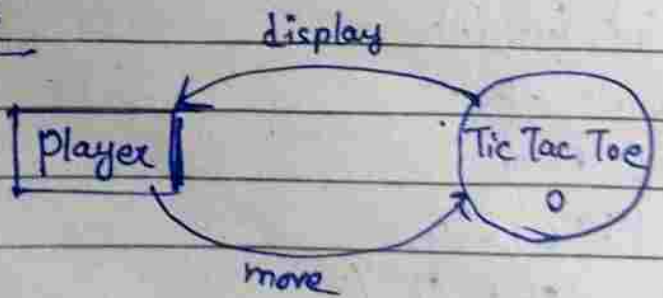
1.1 validate move : → Input: move
→ Output: ~~true, false~~ validated move

1.2 play move : → Input: ~~validated move by computer~~
→ Output: move by computer

1.3 Check winner; → Input: board
→ Output: winner (not yet, player, comp, tie)

1.4 Display : → Input: board
→ Output: Displaying the board.

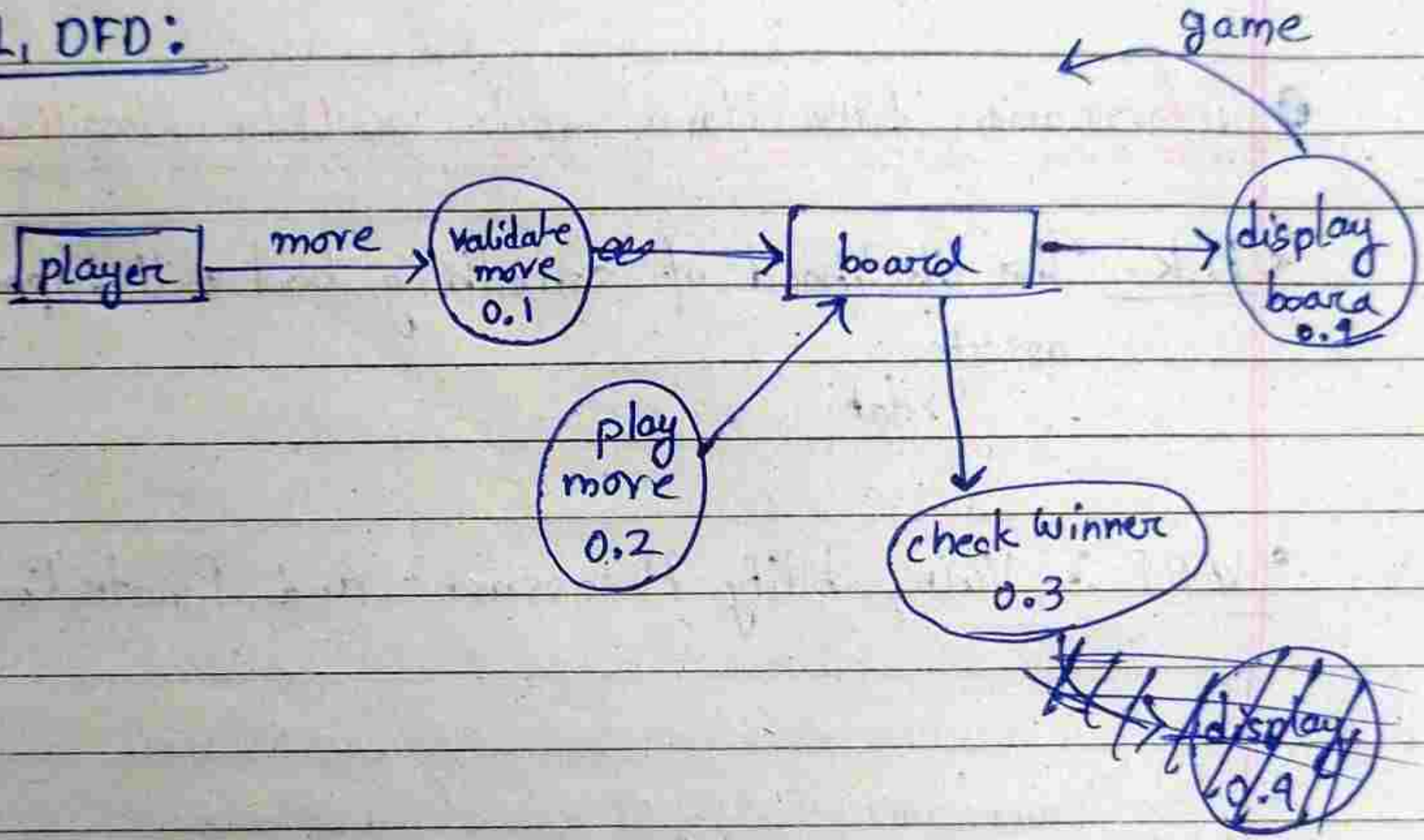
L. DFD:



Date. ____

Page No. ____

L, DFD:



class-69

Software Engineering

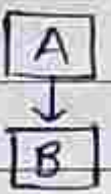
10/10/2025

• Structured Design :

- The aim is to convert the result of structured analysis into structured chart.

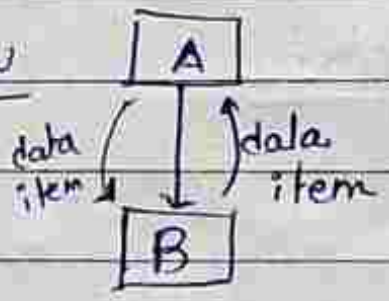
- Symbols used : ① [ ] (Rectangular box) → used to represent module

② Module Invocation Arrow



(module A invokes B)

③ Data flow Arrow



(These are small arrow which ~~does~~ appears alongside Module Invocation arrow.)

- This must be annotated by the comesponding data name.

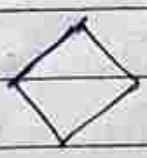
Date.
Page No.

④ Library module :



- If a module is frequently called then we can make that one as a library module..
- It is represented by rectangular shape with double edges.

⑤ Selection :



- Represented by diamond symbol.

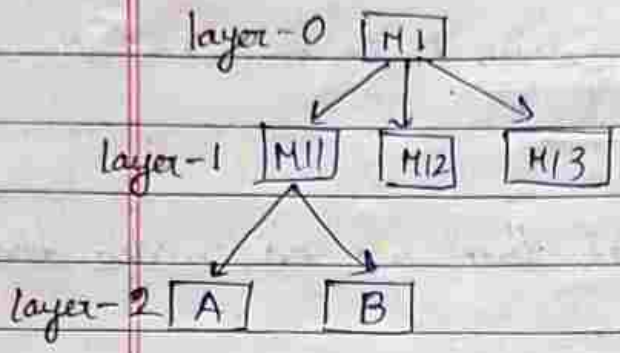
⑥ Repeatation :

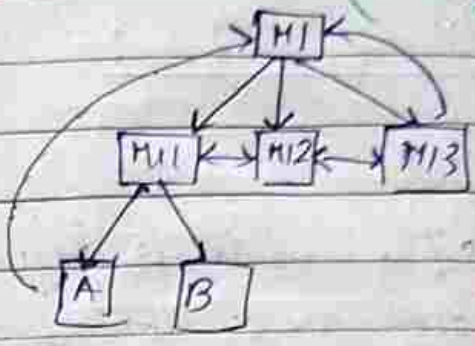
- A loop arrow the control flow arrow denotes the respective modules are invoked repeatedly.
- The structured chart will start with a root module which generally invokes the high level functions or modules.
- If a high level module is decomposible into some modules then all of these could be placed in the subsequent or below levels.

Properly
(Layered design)

Date.
Page No.

Poorly layered design





Transformation of a DFD into structured chart:

Two strategies → ① Transform Analysis
→ ② Transaction Analysis

Transform Analysis:

It identifies the primary functional components that is modules and input & output data

- It divides the DFD into 3 part → Input + Processing + Output
- Input portion of DFD includes, processes that transform input data from physical to logical form and these are known as afferent branches.
- Output portion of DFD transforms output data from logical form to physical form and each of these portion is called efferent branch.
- The remaining portion is called central transform.

Difference is visualising DFD.

(Processes) Takes input from User → Central work → Produces output to the user

Date

Page No

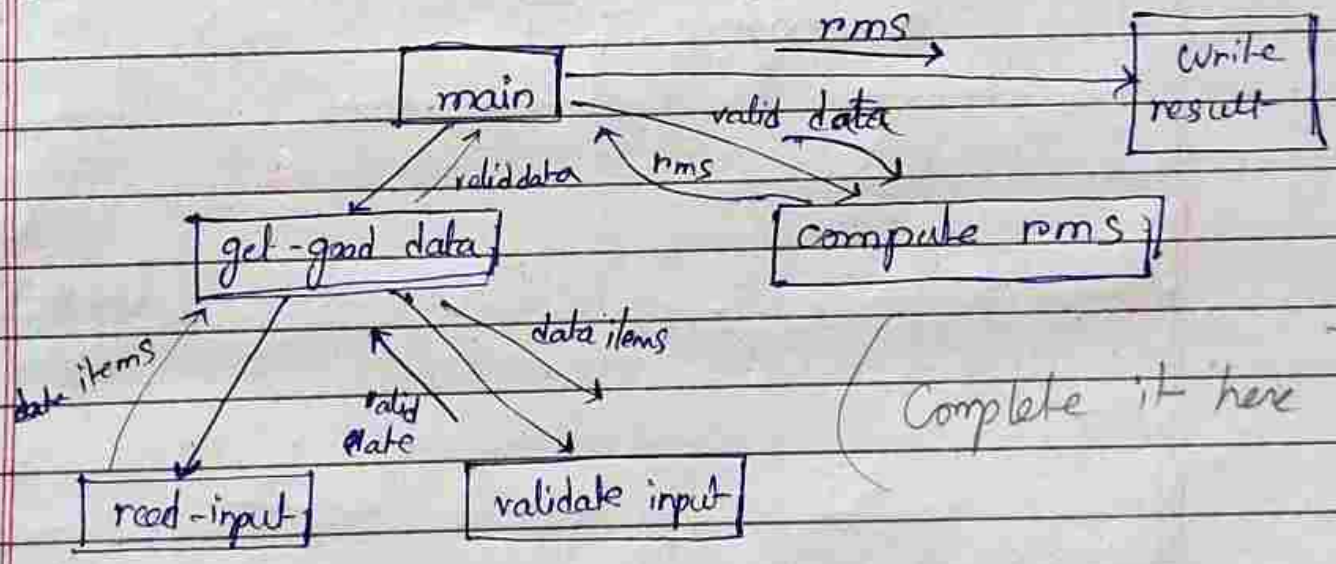
DFD visualizes Transaction

Identify the functions/ processes/ for these 3 steps

- Now the structured chart is derived by drawing one functional component (rectangle) for each central transform; efferent and afferent branch/portions.

- All these functional components must be drawn below a root module

Q. Draw the structured chart of the RMS software.



Transaction Analysis

- It is useful while designing transaction processing programs e.g. – issue book, query book.

- A simple way to identify a transaction is as follows –

Date.
Page No.

① checks the input data.
② the no. of bubbles on which the input data to DFD are incident defines the number of transactions.
③ Some transactions may not require any input data.
④ for each transaction trace the input data to the output.
⑤ Then identify all the bubbles belonging to it.
⑥ Now, draw the root module and below it ~~ident~~ draw each identified transaction as a module.

class-71 Software Engineering Dt. 19/10/25

Unit-III

Object Modelling using UML

- UML → Unified Modelling Language.
- Analysis Model:
- Design Model:
- Design Methodology:

- A model of a given problem is called its analysis model on the other hand a model of a solution or code is called design model.

- The design model is obtained by carrying out iterative refinements to the analysis model using a design methodology.

- Design: Converting some specification into architectural form.

Date
Page No.

• Modeling language versus Design Process:

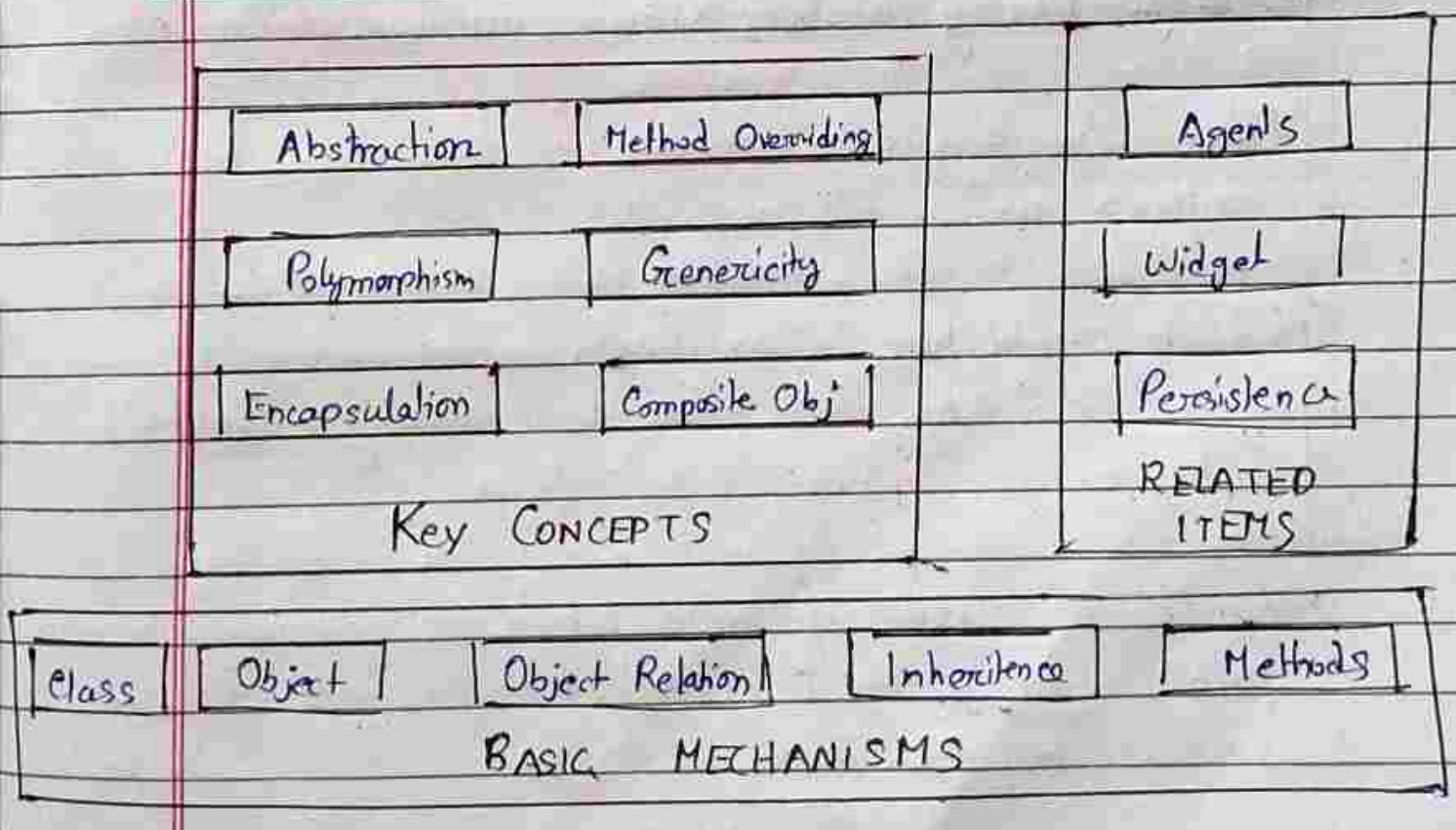
• Modelling language

- It consists of a set of notations using which design and analysis models are documented. e.g. UML

• Design Process

- Given a problem description, how to systematically work out the design solution to the problem?
- It is the step by step procedure using which a problem description can be converted into a design process.

Basic OOP concepts:



Date.
Page No.

- Polymorphism → Many forms of an object.
- Inheritence → Object can inherit characteristics from other classes.
- Encapsulation →

BASIC CONCEPTS :

① Objects : → Tangible real world entities.

→ It has some properties (variables / attributes) and functionalities (methods / processes).

→ It gets you high cohesion and low coupling.

→ eg.: library member, a book, an issue register, etc.

→ It is advantageous to consider certain conceptual entities as objects as well.

→ Objects would help in excellent decomposition of the system into parts that have low coupling and high cohesion.

→ An object consists of data which is generally marked private and functions / methods / operations that operates on this data.

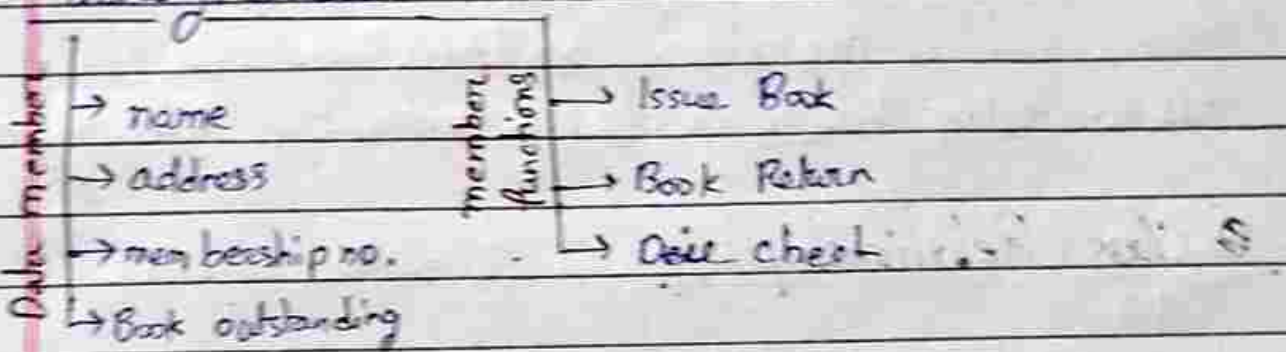
Date
Page No.

- **Data hiding:**

If the data are made private, no objects can directly access the data of any other object. This mechanism of hiding data from other objects is popularly known as principle of Data hiding. It promotes high cohesion and low coupling.

- **eg: Library Automation Software**

- **Library Member:**



② Classes:

- Similar objects constitute a class hence they have similar attributes and methods.

- **Abstract Data Type (ADT):**

- All classes are ADT but not vice versa.

- Two things are inherent to ADT, one is Abstract Data and the second is Data type.

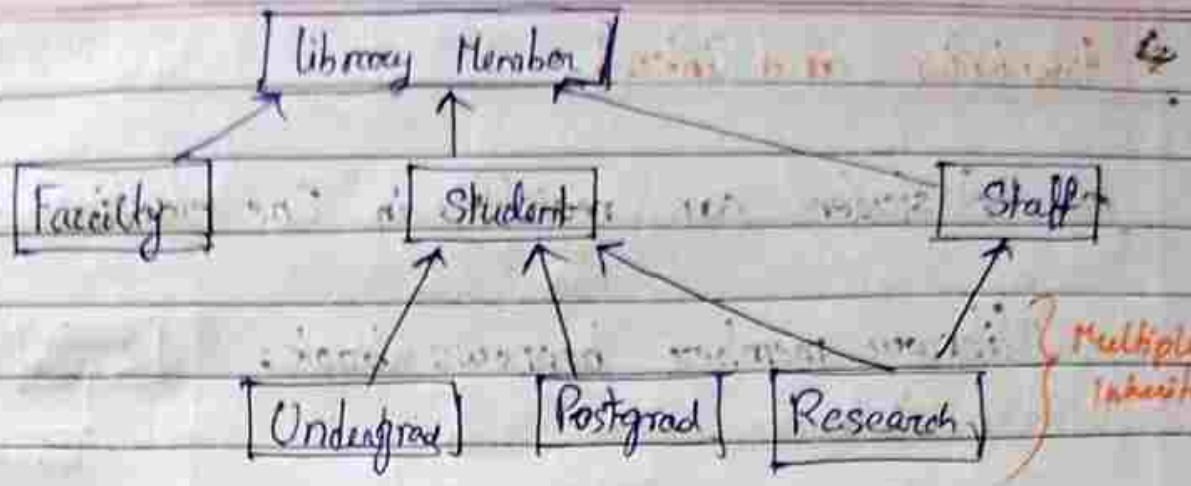
Date
Page No.

- **Abstract Data:**
  - Data of an object can be accessed only through its methods.
  - The exact way data is stored internally, i.e. stack, array, queue, etc in the object is abstracted out; i.e. not known to other objects.

- **Data Types:** A data type defines a collection of data values and a set of predefined operations on these values.
- It is instanciated to create a variable of that type.

- **Class Relationships:**
  - Inheritence
  - Association & link
  - Aggregation & Composition
  - Dependancy.

- **Inheritance:** It allows one to define a new class by extending the features on an existing class. The original class is known as baseclass/ superclass / parent class.
  The new class if called as childclass/ derived class.



- In this example, a faculty member is a special type of library member and the relation is called "is a" relation.

- method overriding:

A derived class may even redefine some methods which are already existing in the base class. This is known as method overriding.

- The base class method is said to be overridden in the child class.

Advantages: code reuse.
simplicity of program design

- Multiple Inheritance:

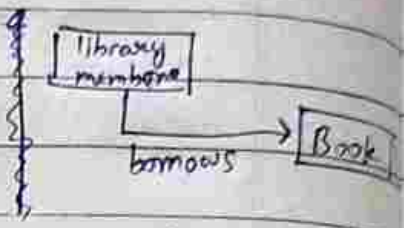
- One class inherits data and methods from multiple classes.

Date.
Page No.

Association and links:

ex: ① A student can register in One elective Subject.
link

② library member borrows book.
link



- ~~link is~~ It is a common relation among classes and frequently occurs in design solutions.
- When two classes are associated they can take each other's help by invoking each other's methods.

ex. A student can register in 1 elective Subject.

→ An elective subject would know IDs of all student objects that have registered for that subject.

ex. Library members borrows books.

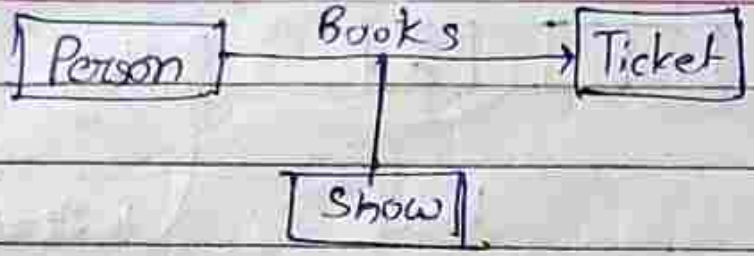
↳ It represents a bidirectional association.

N-ary association:

- 3 or more different classes are involved in an association.

ex. A person books a ticket for a certain show.

Date.
Page No.



Composition & Aggregation:

- It ~~repne~~ Objects which contain other objects are called composite objects.

eg. A book can have upto 10 chapters.
book → object
chapters → objects
↳ this is subobject / sub class

↳ A single book object is an aggregate of 1 to 10 chapter objects.

class-76

Date 17/10/25
Page No.

Software Engineering

Dependancy :

Dependancy b/w two classes are as follows —

① A method takes an object of another class as an argument.
② A class has an object of another class as its local variable.

ABSTRACT CLASS :

- Classes that are not intended to produce instances of themselves are called abstract classes.

- How to identify the class relationships ?

- for composition → B is a permanent part of A.
  A is made up of B's.

- Aggregation → B is a part of A.
  A contains B.

- Inheritence → A is a kind of B.
  A is a specialisation of B.
  A behaves like B.

Date.
Page No.

- Association → A needs helps from B
  A collaborates with B

Related Technical Terms

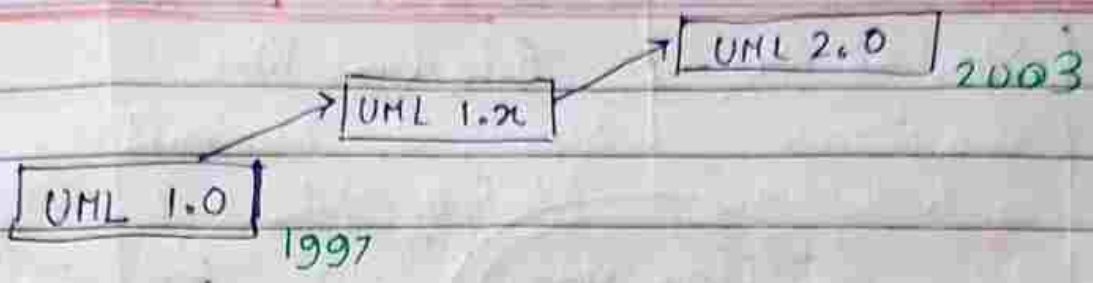
- **Persistence:** Objects are stored permanently
- **Agent:** An agent monitors events such as Keyboard press and action taken.
- **Widgets:** It stands for window objects, which is a primitive object used for GUI design. It maintains internal data such as Geometry of window, and BG colors or foreground colors, cursor shape and size etc.

UML and UML Diagrams:

- **Model:** ~~Structure of~~
  A model is an abstraction of real world problems or situation and is constructed by leaving out unnecessary details.
- **UML:** It is an Unified Modeling language used to document the modal such as analytical and design.

Date _____
Page No _____

Evolution of UML :



Q. Why to construct a model?

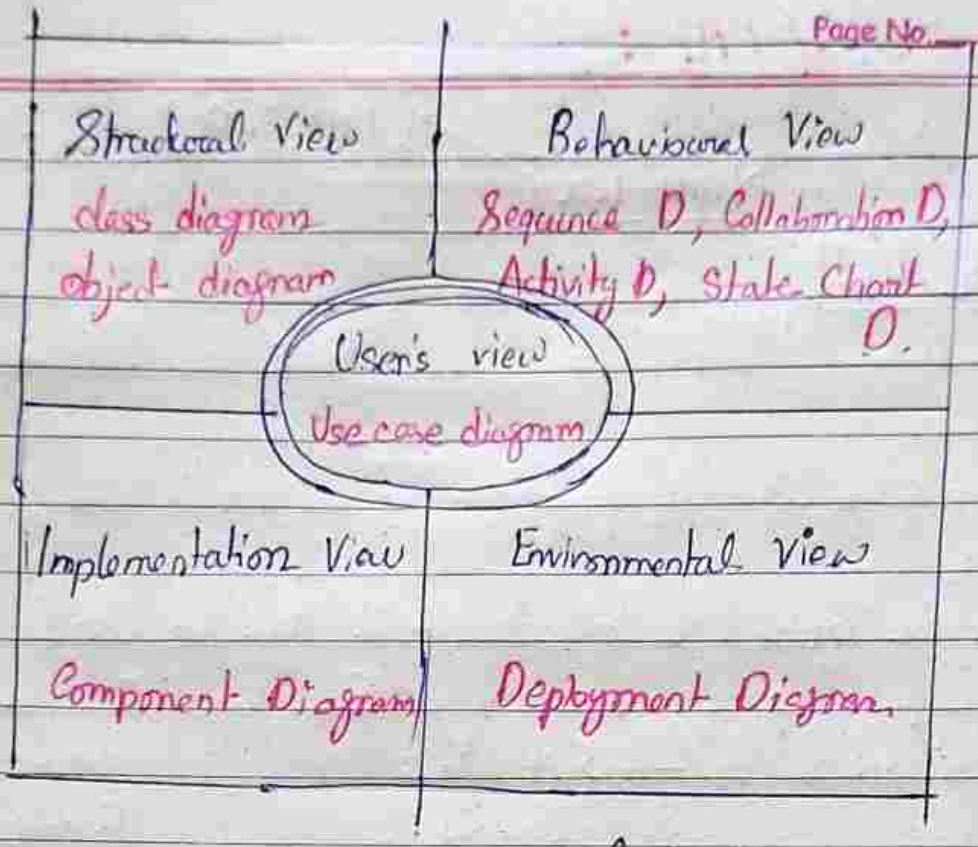
→ Once a model is created, it can be used for a variety of purposes during Software Development Process such as → Analysis, Design, Specification, Coding, Testing, Visulisation etc.

UML Diagrams :

- If a single model is constructed, then it would become very complex and difficult to understand.
- So UML provides several diagrams which helps to capture the following 5 views of model —

(a) Users view
(b) Structural view
(c) Behavioural View
(d) Implementation View
(e) Environmental View

Date.
Page No.



- User view ⟶ DFD-0 like (BBox)
- Structural view ⟶ define classes and objects and functionality
- Behavioural View ⟶ flow and relationships (9 types).
- Environmental View ⟶
- Implementation View ⟶ GUI parts used.

- It captures the functionalities offered by the system, it is a black box view of a system.
- It defines the structure of problem or solutions in terms of Objects and classes.
- It captures how objects interact with each other with time
- It captures the important components of the system and their inter-dependancy.

Date. ____

Page No ____

Environmental View:

This view models how the different components are implemented on different pieces on hardware.

Software Engineering

Date 31/10/2025
Page No

① Use case modeling / Diagram :

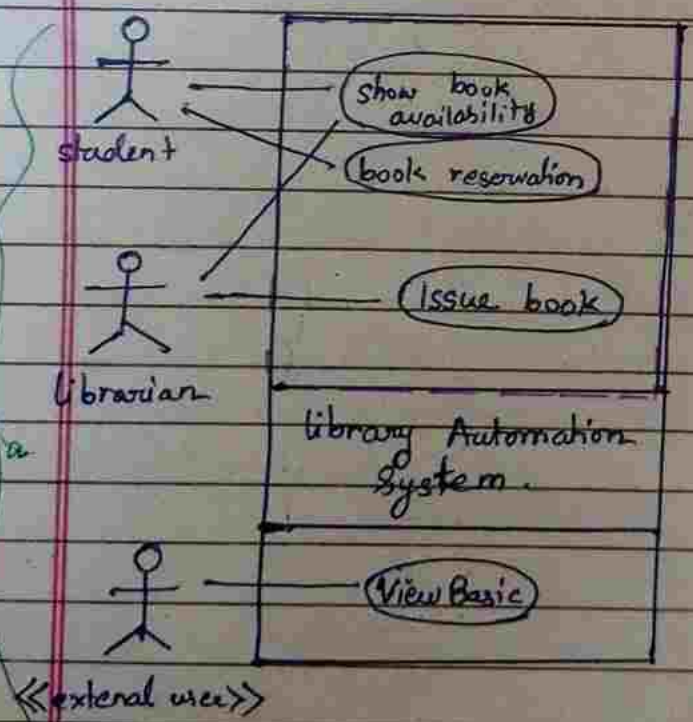
Captures functionalities/services that the system provide to user.

↳ High level functions / sequence.

→ Service is represented by ellipse. (—) along with name.

→ Users are represented by 🧍 along with a label

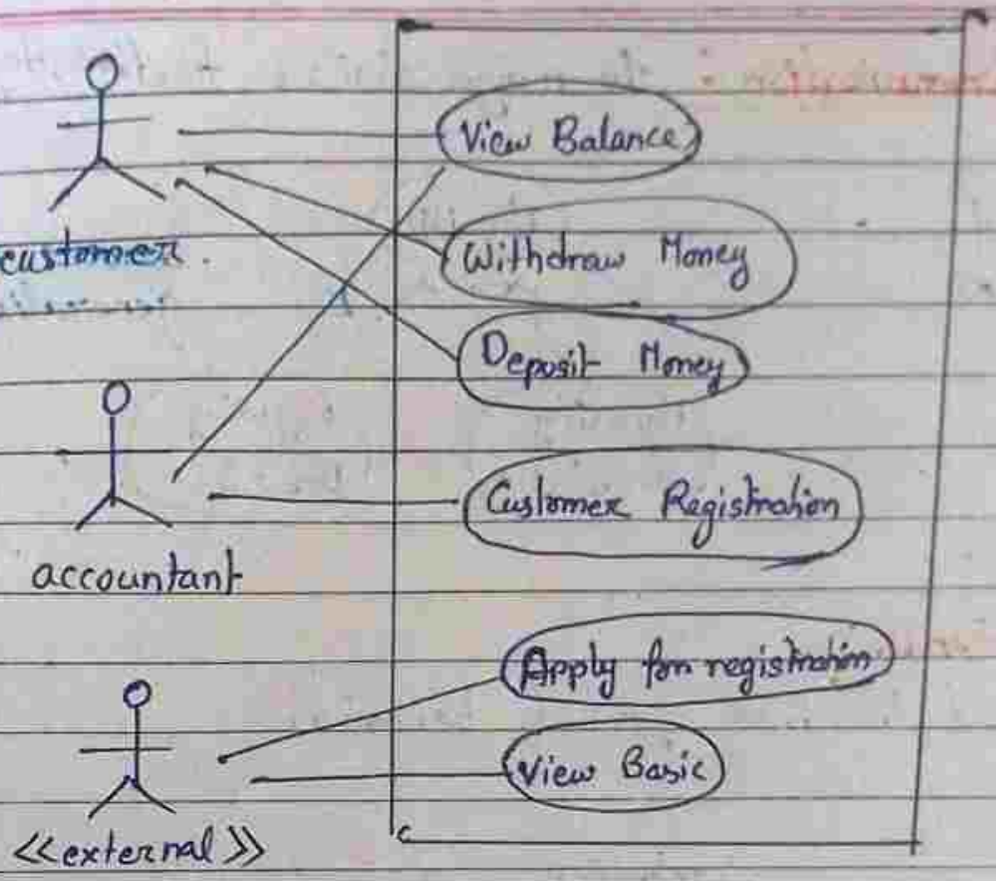
↳ for external user <<external>> stereotyping.



Q. Create a use case diagram for accounts and customer of a banking automation system.

Date
Page No

CAN BE DIFFERENT/ MORE ANSWERS.



- Give loan / update customer info. / view customer info.

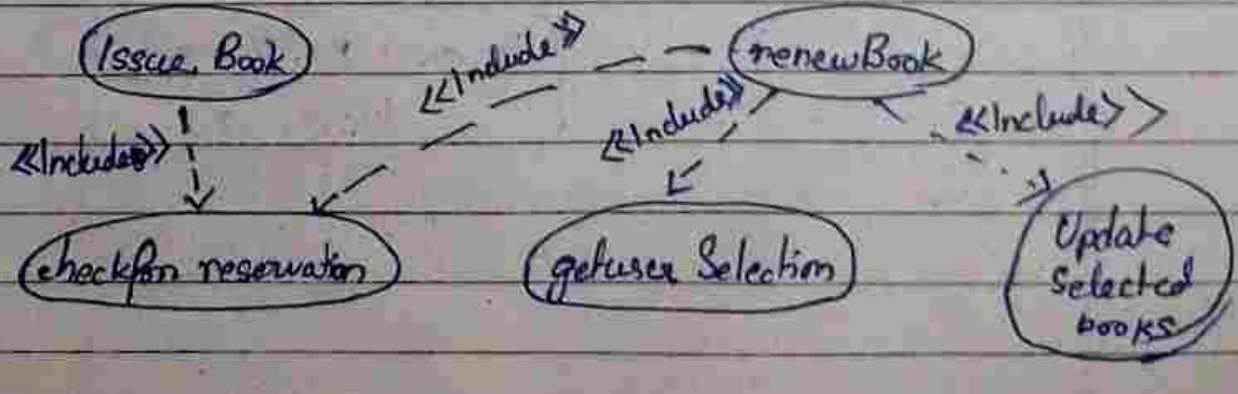
→ UML provides stereography to write meaningful things.

Relationship among Use cases :

(1) Include :

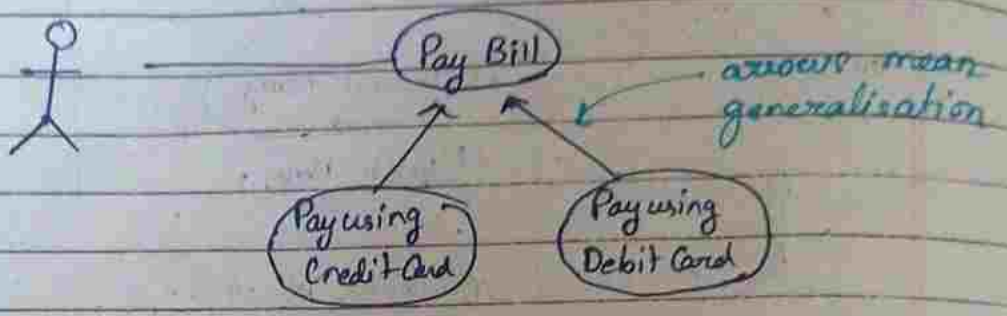
1 or more use case includes the behaviour of another use case.

ex.



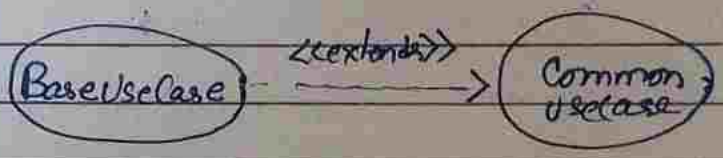
Date
Page No

(ii) Generalisation : to merge similar functions together



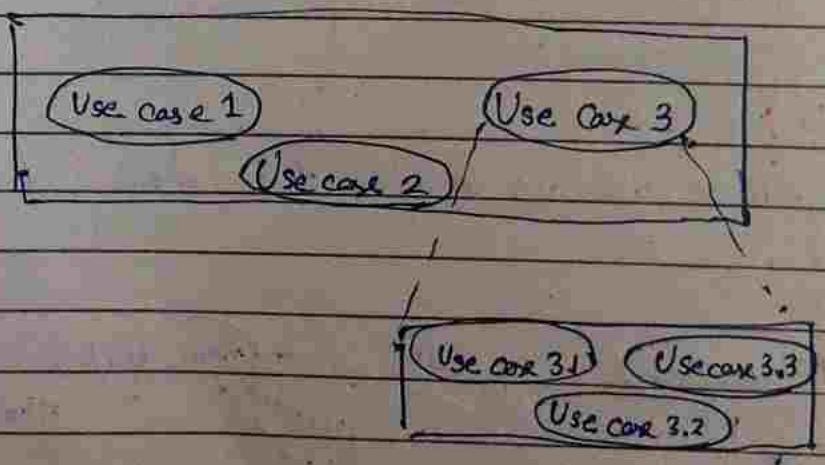
(iii) Extends :

used to show optional behaviour



(iv) Organisation :

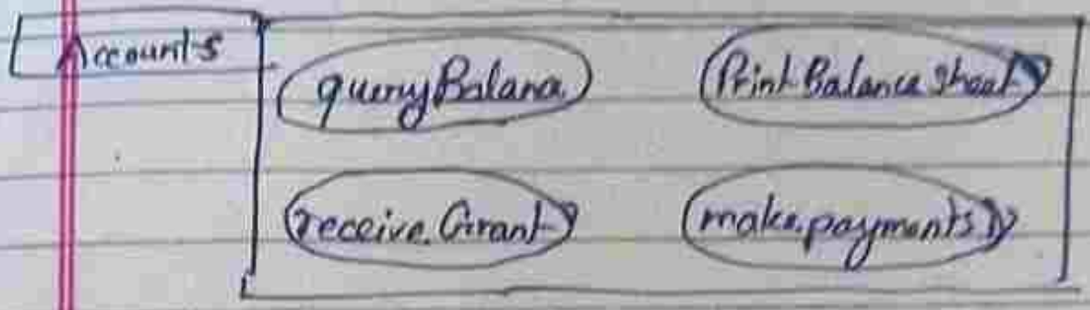
hierarchy representation



Date
Page No

(v) Use case packaging:

Similare use cases can be packaged

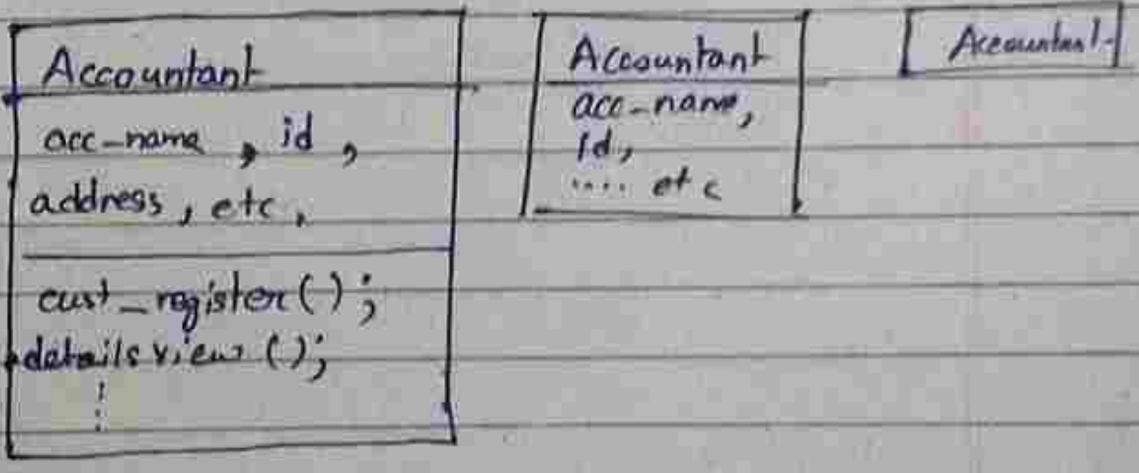


(2) Class Diagrams:

It defines structure of the model.

(i) Class: class is identified by a Name. And name will always start with capital.

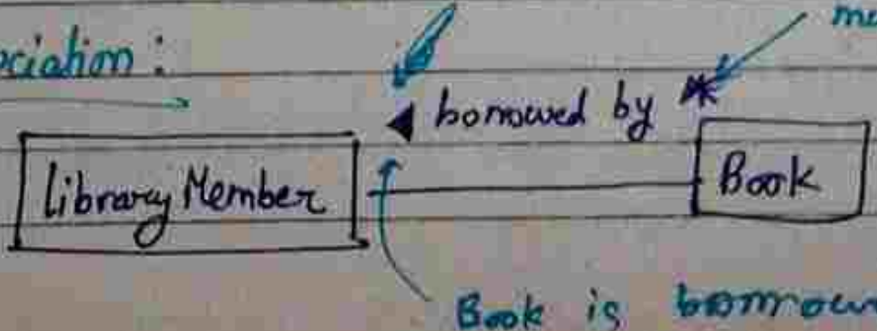
ex:



(ii) Attributes

Properties of class

(iii) Association:



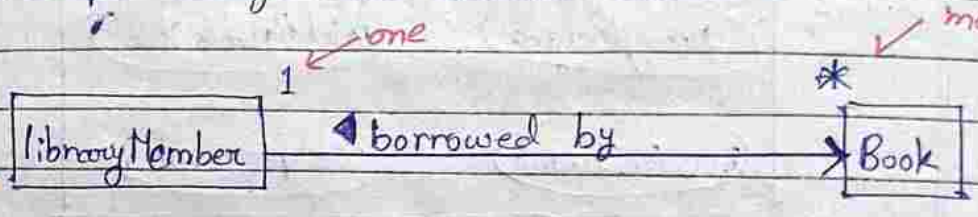
class -79

Software Engineering

Date 03/11/2025
Page No.

(iii) Association :

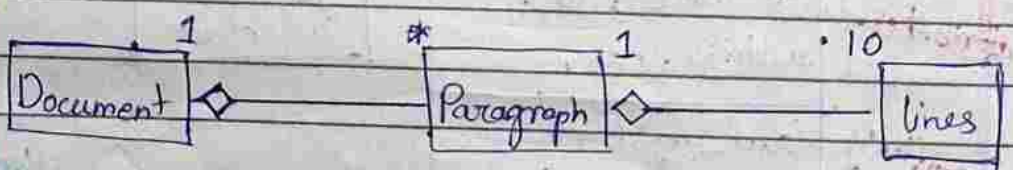
uses simple straight line between classes.



- Book (is) borrowed by libraryMember.
- Number above class box implies number (for many to one (one to many)

Many books may be borrowed by a library member.
OR, A book may be borrowed by a library member.

(iv) Aggregation :



- A document can be considered as an aggregation of paragraphs. Each paragraph is an aggregation of lines (upto 10).

(v) Composition :



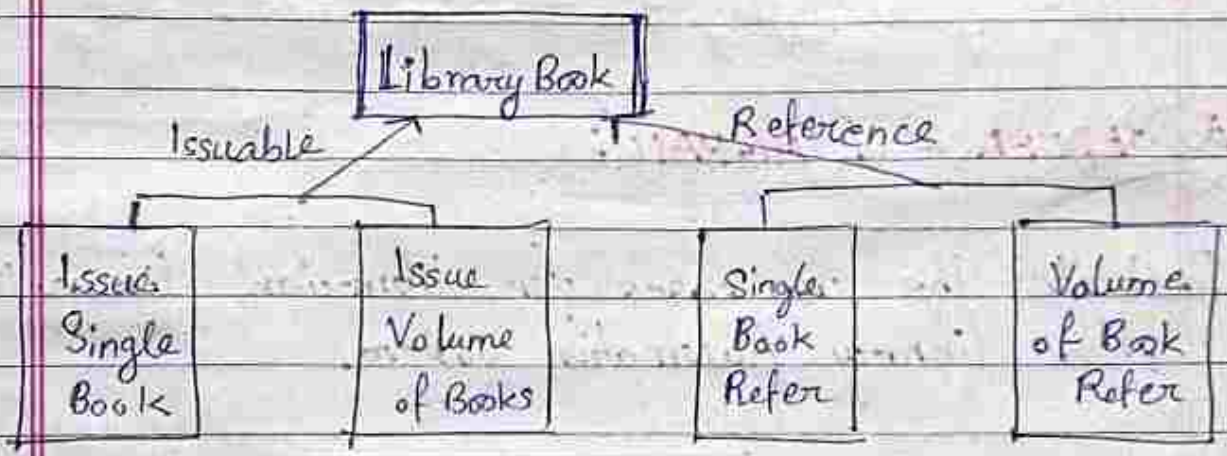
- If any change in items, order needs to drops.

Date. _____

Page No. _____

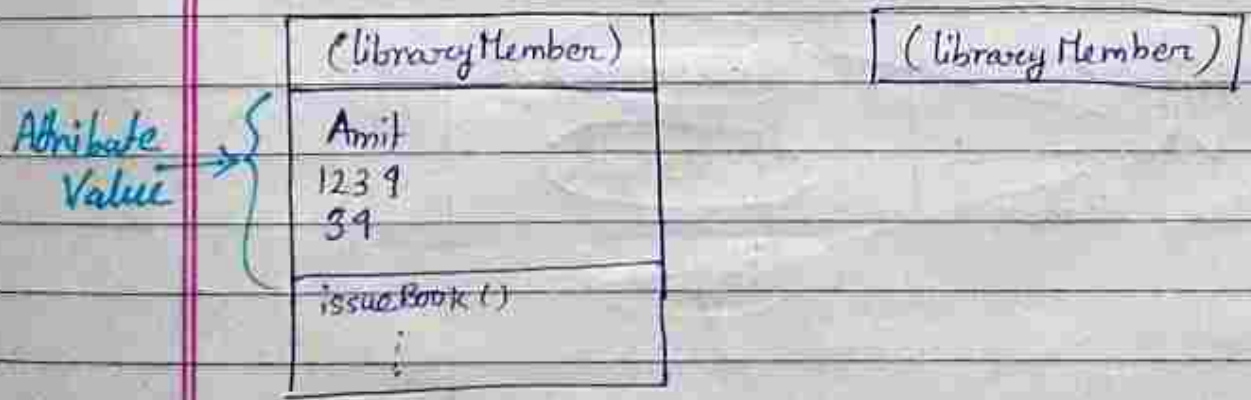
- Order is composition of items, but here after placing the order, no items in the order can be changed.
- It is the stricter version of aggregation.

(VI) Inheritance :



- [Constraint] ← constraints are written like this.

OBJECT DIAGRAM



Date.
Page No.

(3) INTERACTION DIAGRAM :

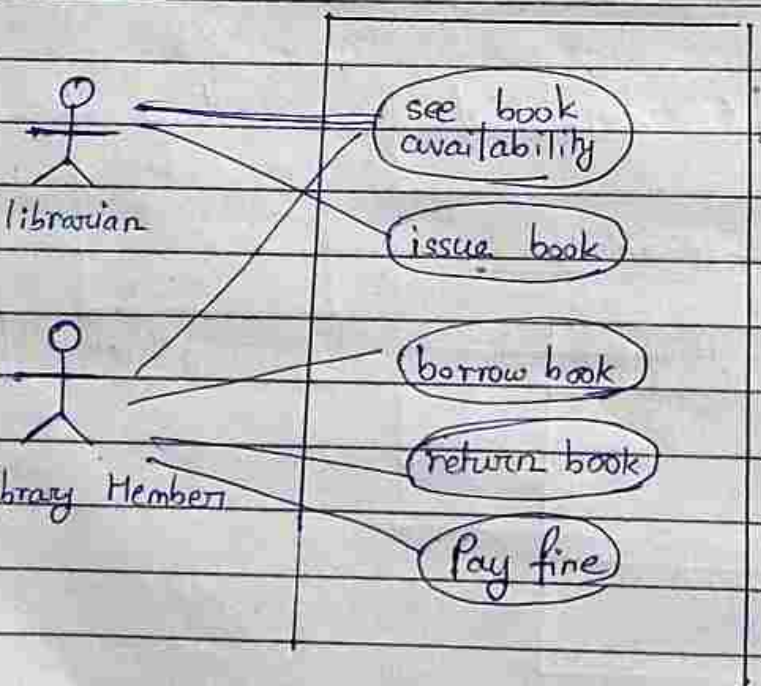
- Diagram of interaction b/w group of objects.
- There is 2 types - (i) Sequence.
  (ii) Collaboration.

- Works by identifying the functionalityies.

(3a) SEQUENCE DIAGRAM :

Q. Create the use cases for librarian, libraryMember of a library Automation System.

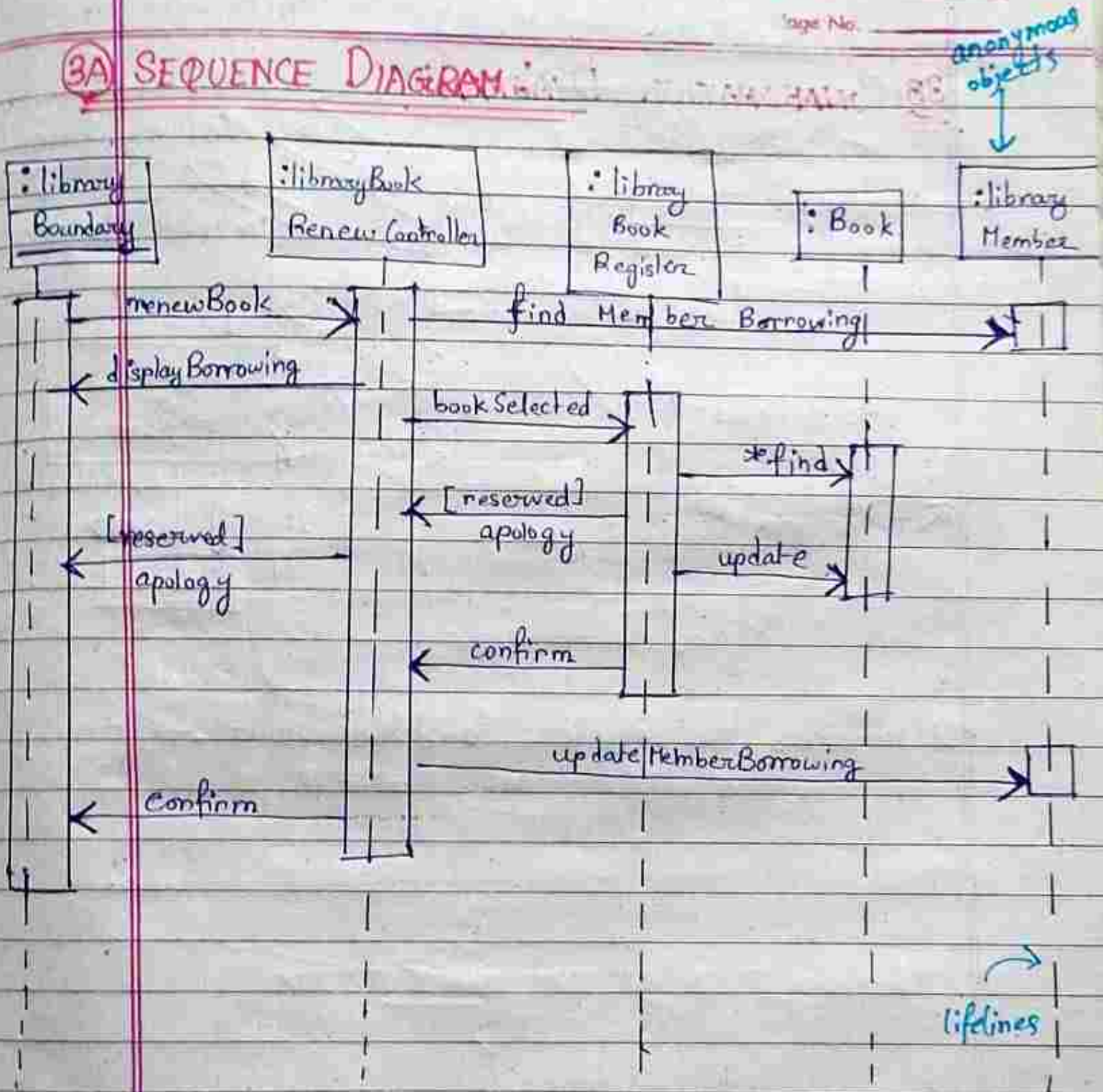
Ans.



Date

Page No.

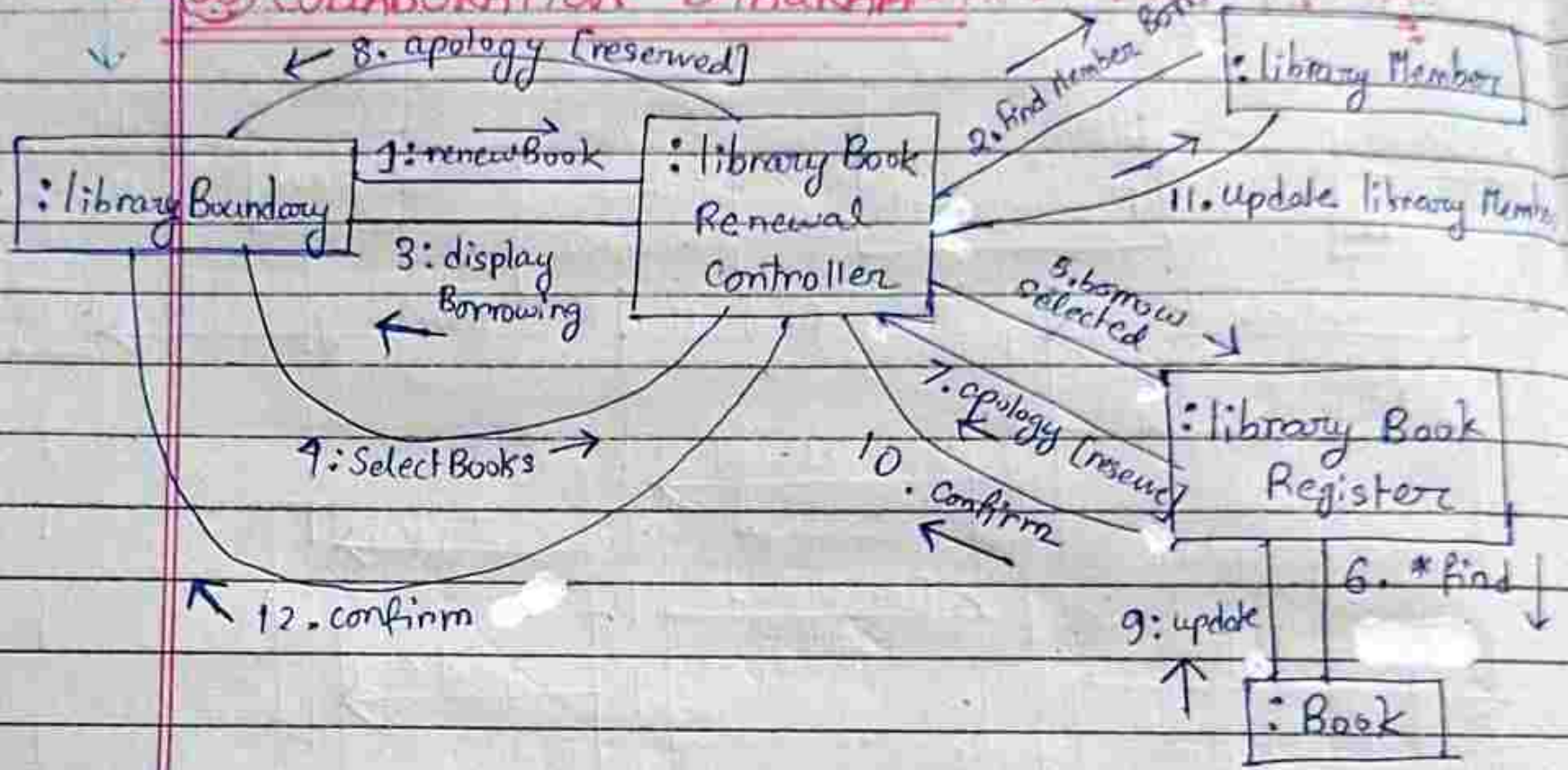
3A) SEQUENCE DIAGRAM



Date _____

Page No _____

(3B) COLLABORATION DIAGRAM



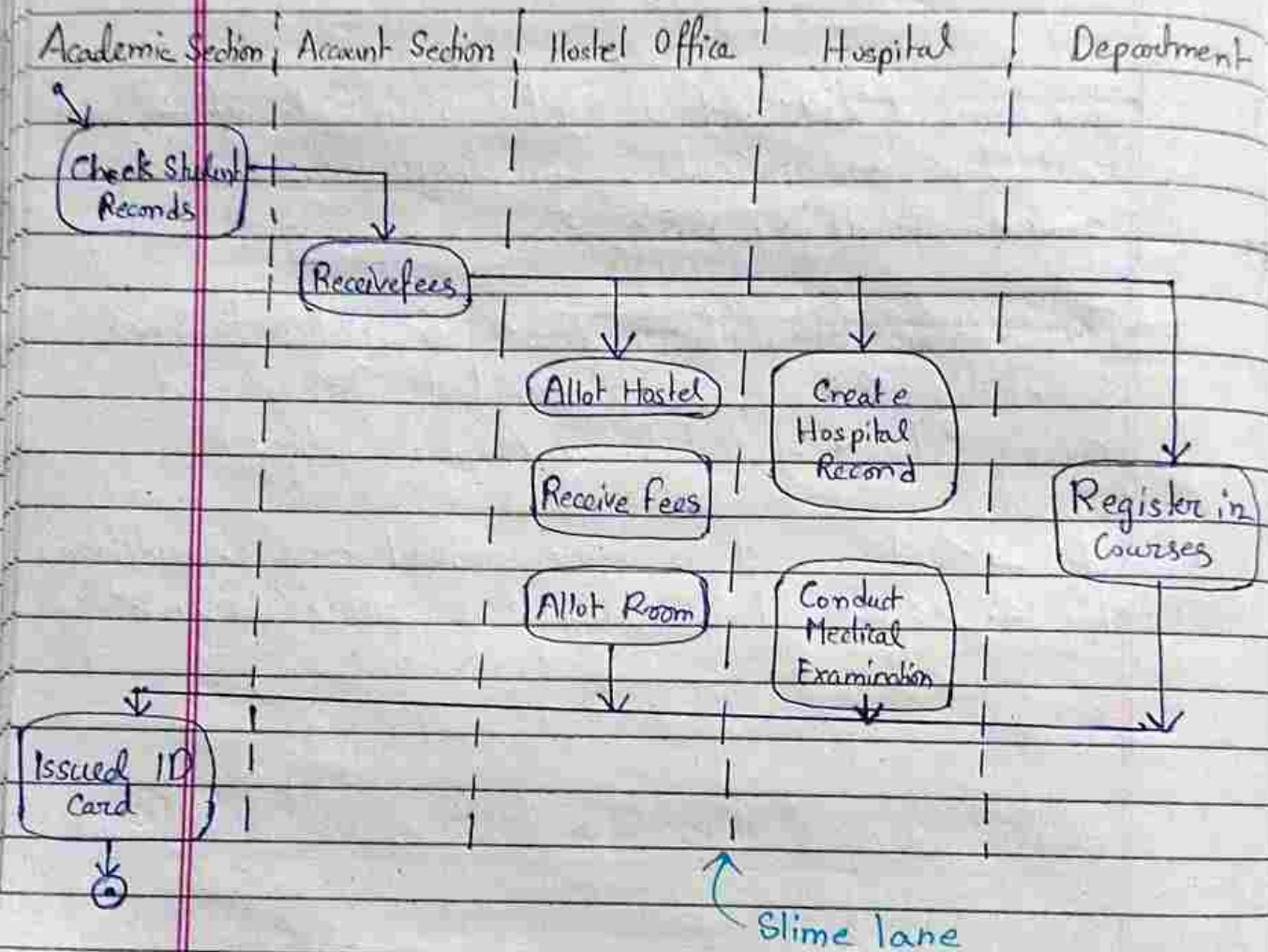
- It decribes the structure and behaviour.

Class-81 Software Engineering

Date 09/11/2025
Page No

④ ACTIVITY DIAGRAM:

- Classes will be written on the top of the sheet.



- It is similar to a flow chart.
- An activity may not correspond to methods of classes but they represent higher granularity tasks.

Date
Page No

⑤ STATE CHART DIAGRAM :

- It is normally used to model, how the state of an object may change over its lifetime.

- The state represents behaviour.
- The lifetime means use-case execution.

- State Chart Diagram has 4 components —

① Initial state : It is represented by a filled circle.

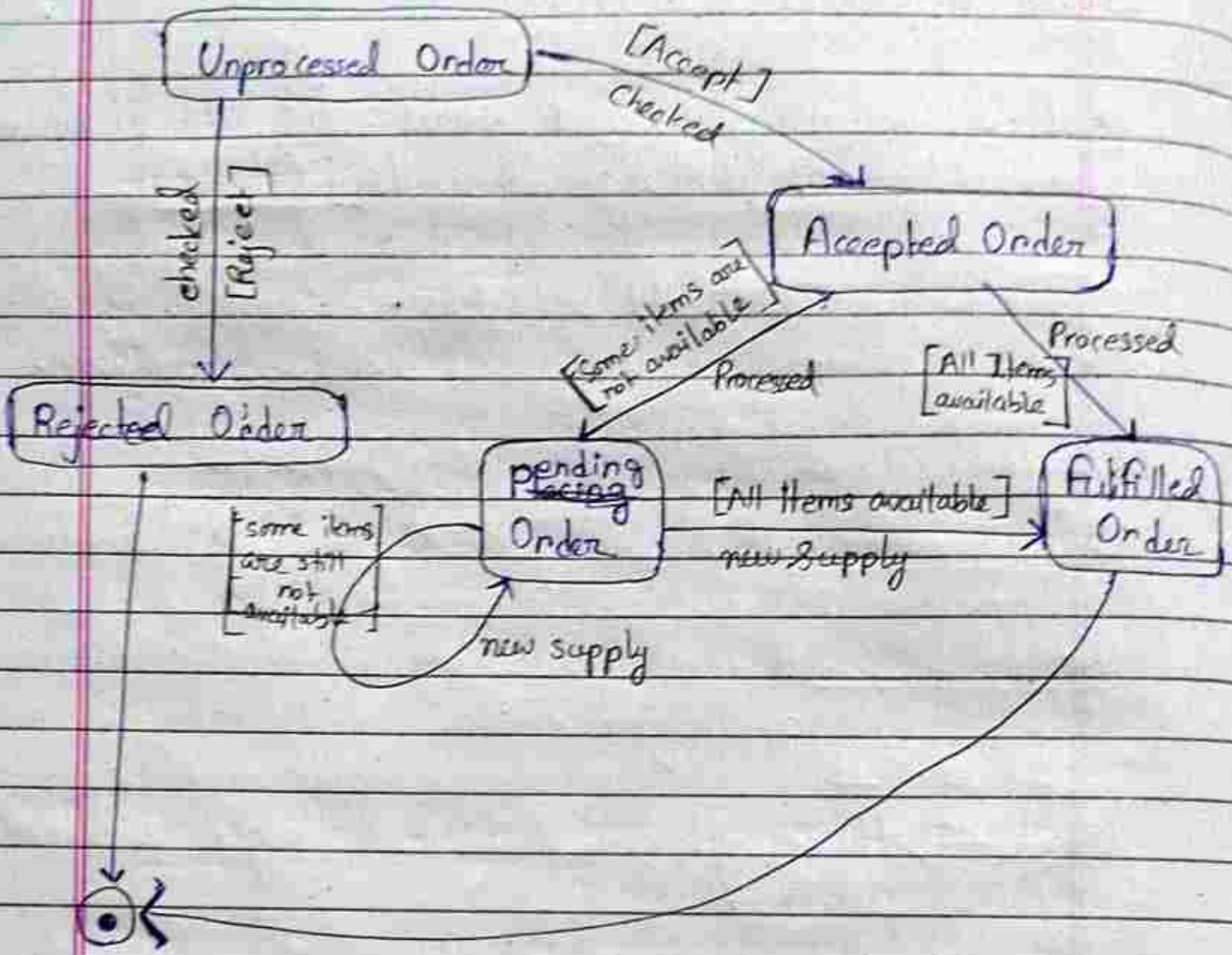
② final state : It is represented by a filled circle inside a larger circle.

③ state : It is represented by a rectangle with rounded corners.

④ Transition : It is shown by an arrow between two states.

Date _____

Page No. _____



OBJECT ORIENTED SOFTWARE DEVELOPMENT

- It consists of 2 things —

  (A) Object Oriented Analysis
  (B) Object Oriented Design.

- The term object oriented analysis refers to developing an initial model of a software product from analysis of its requirements & specifications.

Date
Page No.

- The analysis model is very difficult to translate into code whereas a design model can be easily transmitted to code.

- The generic methodology for developing OOD consists of
  - Use case Model
  - Domain Model
  - Design Model

  (Use case Model and Domain Model are Iteratively Refined)

PATTERNS :

- Patterns are building blocks of design.

Design Patterns :

- Documented
  - Problem
  - The context in which the problem occurs
  - Solution
  - The context in which the solution would or would not work.

Types of Patterns
- Architectures
- Design → class/object/attribute role/collaboration
- Idioms

↓

low level implementation detail.

Date
Page No

② Some Common Design Patterns

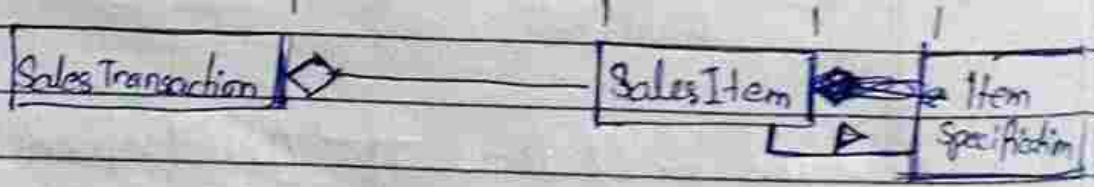
① Expert
~~② Facades P~~
② Facade Pattern
③ Creator Pattern
④ Model View Separation Pattern
⑤ Intermediary or proxy pattern
⑥ Observer pattern
⑦ Multiview Controller Pattern
⑧ Publish-Subscribe Pattern

Expert Pattern:

- Problem Addressed: when a certain activity needs to be performed which class should be assigned the responsibility for doing it?

- The expert pattern expresses the common intuition that the object should do things related to the information they store.

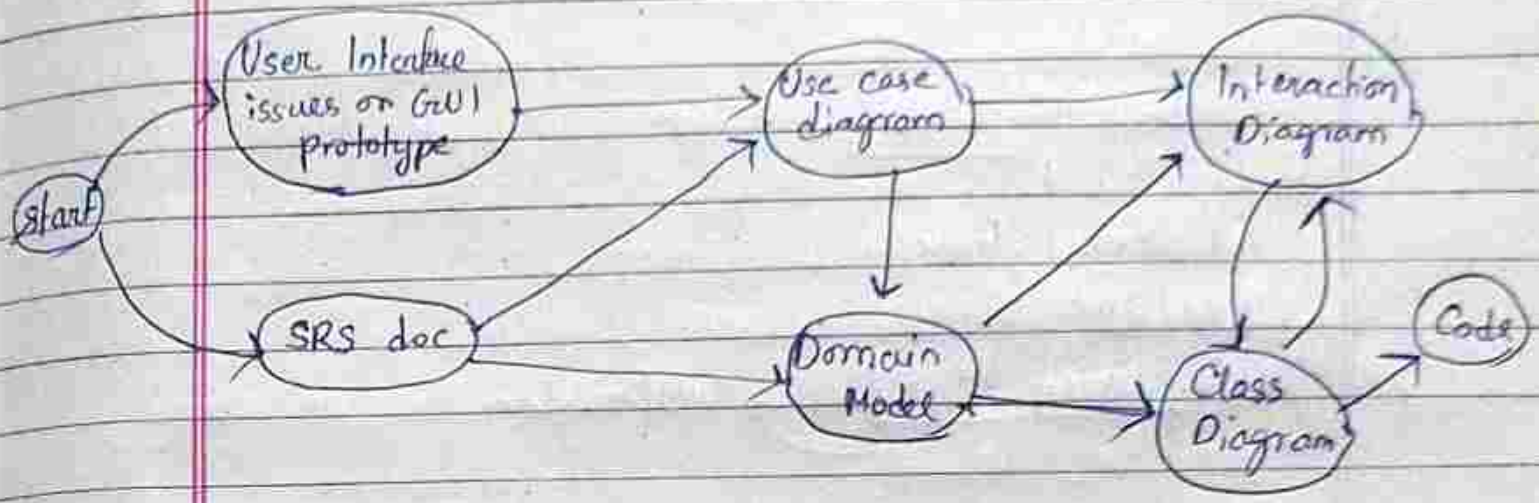
> Ex.



Date
Page No

OOA and OOD Methodology

(OOAD)

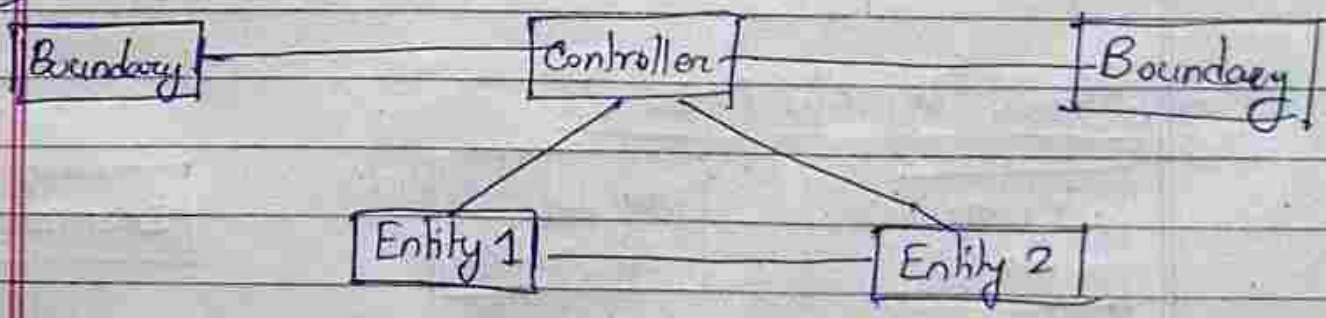


Domain Modeling :

It is also called conceptual modeling.
object

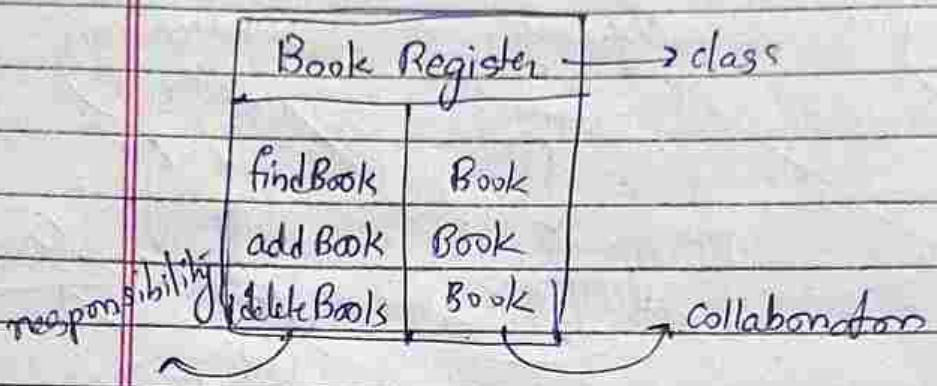
objects can be 3 types.
- ① Boundary Object → Interfaces (input/output/validation)
- ② Controller Object → Collaboration (co-ordinate activities)
- ③ Entity Object → Store the information

ex.



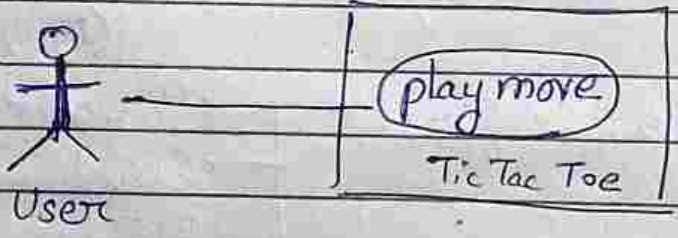
Date
Page No

Class Responsibility Collaboration Card (CRC Card)



Q. Using OO Paradigm, find the analysis, domain and design Models for Tic Tac Toe Problem.

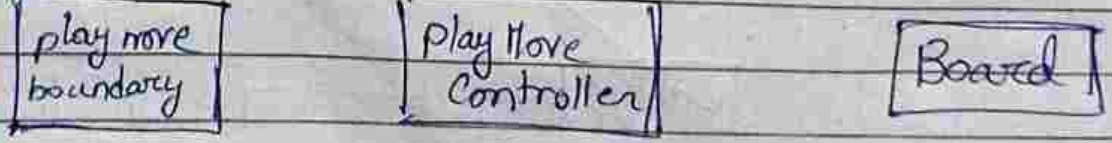
Ans. ① Use Case Diagram



② Initial domain Model



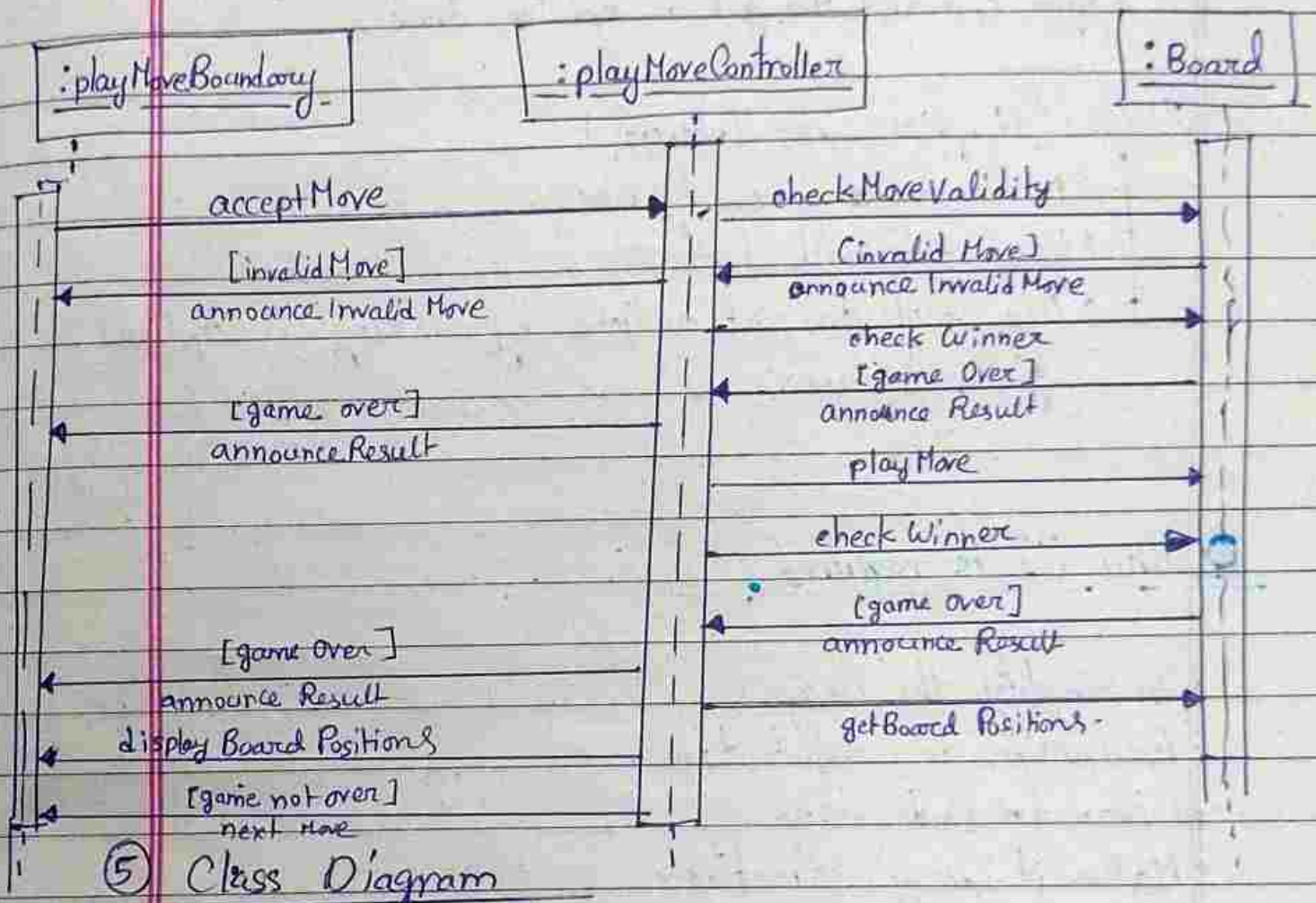
③ Refined Domain Model



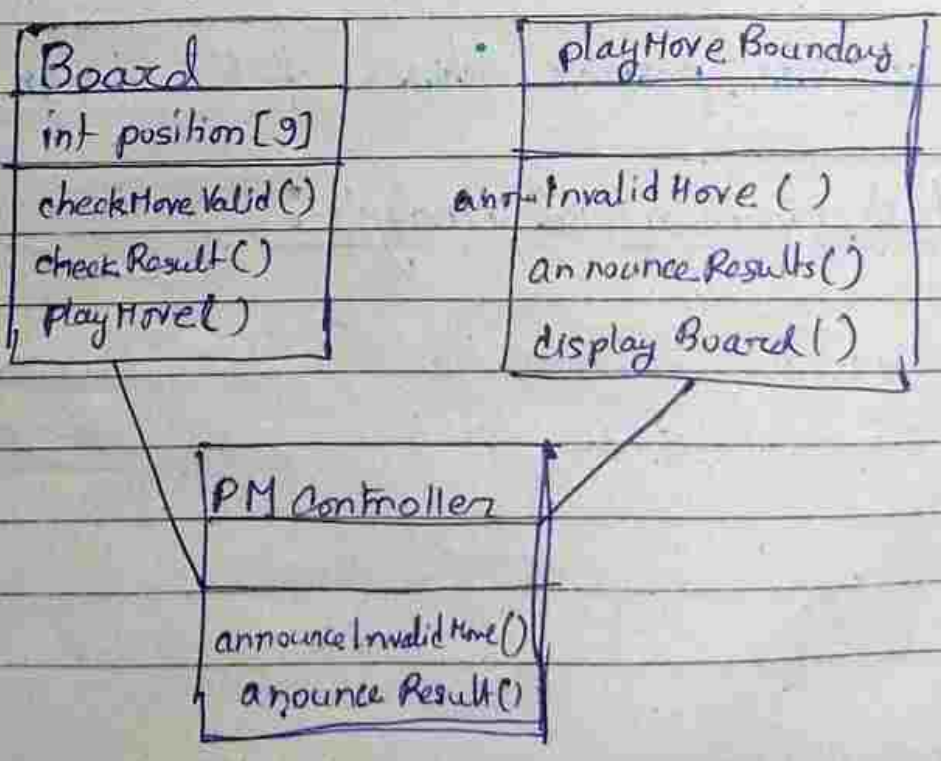
(Arrow not compulsory)

④ Interaction Diagram (i.e. Sequence Diagram)

(among the entities identified in refined domain model).



⑤ Class Diagram



class-82

Software Engineering

Date 07/11/2025
Page No.

USER INTERFACE DESIGN:

① CLI (command line Interface.)
↳ give command to get a service done.

② GUI (Graphical User Interface)
↳ No command given by user.
↳ Uses text with buttons.
↳ User will do set a jobs by clicking on options given to user.

options → buttons / hyperlink

Why UI is required?

- To simplify the usage.
- Responsive in short time.
- Clear to understand.
- Make it very attractive.

What are the possible phases of UI design process?

① User, task, environmental Analysis:

⇒ The focus is based on the profile of users who will interact with the system.

② Interface Design :

⇒ Define the set of interface objects and their actions.

③ Implementation :

⇒ Creation of a prototype or model that enables usage scenarios to be evaluated.

④ Interface Validation :

⇒ Deals with the testing of the Interface.

Q. A super market needs to develop a software that would help it to automate a scheme that it plans to introduce to encourage regular customers. In this scheme, a customer would have first register by supplying his or her residence address, telephone number and the driving license number; each customer who registers for this scheme is assigned with a unique customer number by the computer. A customer can present his customer number to the checkout staff when he makes any purchase. In this case the value of his purchase is credited against his CN (customer no.) at the end of each year the Supermarket intends to award surprise gifts to customers

Date.
Page No.

who make the highest total purchase over the year. Also, it intends to award a 22K gold coin to every customer whose purchase exceeded 10,000 rupees. The entries against the CN are reset on the last day of every year after the prize winner's lists are generated.

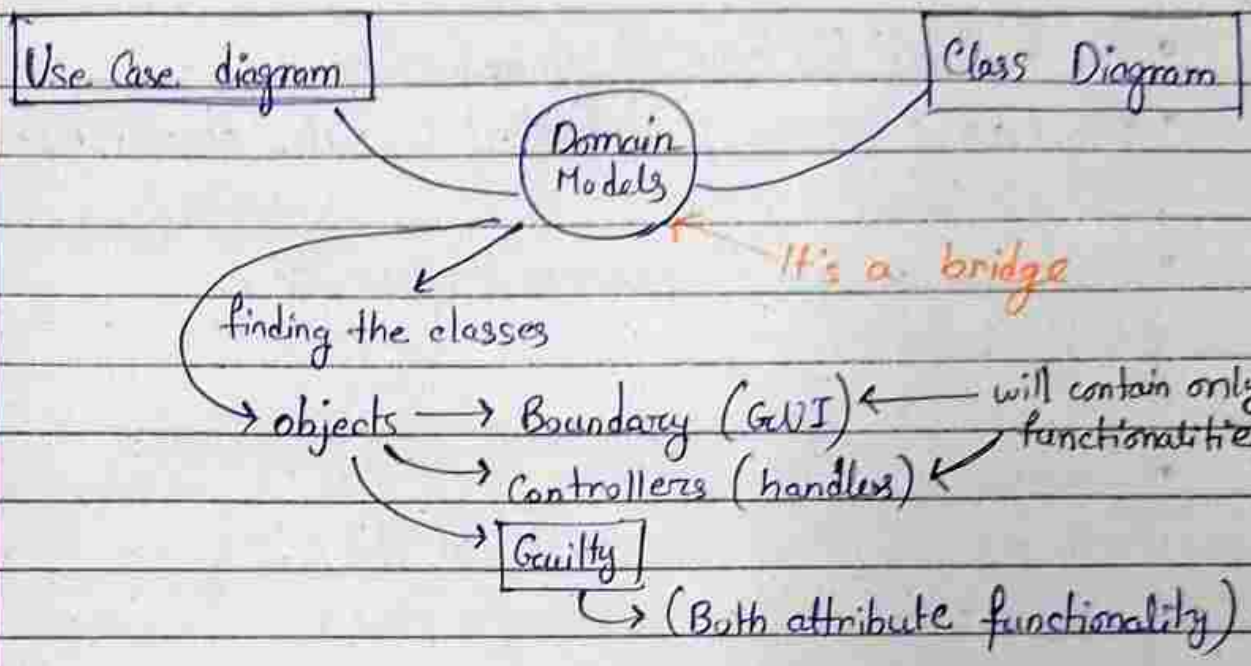
- Apply OOD methodologies to obtain —
  (A) Use case diagram
  (B) Domain model
  (C) Interaction Diagram (Sequence)
  (D) Class Diagram & its association.

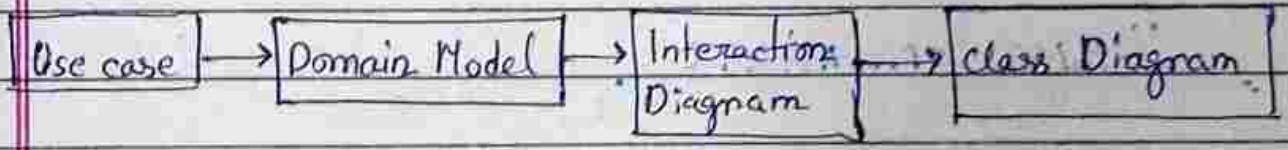
class-83

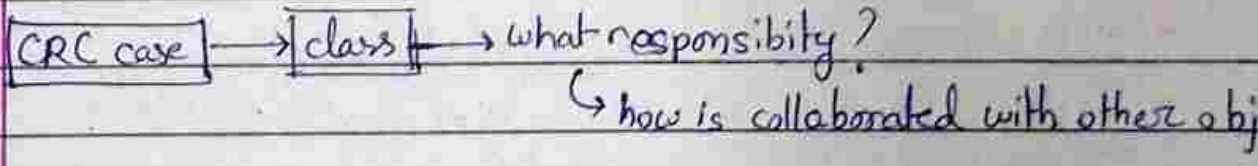
Date 14/11/2025
Page No.

Software Engineering

UNIT IV : Coding & Testing







CODING : (Converting Design Attributes/aspects into some sort of executable programs (.exe))

- We need to follow some sort of coding standards in order to achieve following advantage :—
  1. Uniform appearance of the code.
  2. Enhance the readability/understandability/reusability of the code.
  3. Achieve good programming practices.

Date.
Page No.

CODING STANDARD & GUIDELINES

- Try to limit the use of global data.
- Standard headers of different modules should include module name.
- Data on which the module was created.
- Author's name.
- Modification history.
- Synopsis of the module.
- Support functions along with i/p & o/p parameters.
- Global variable accessed or modified by the module.

NAMING CONVENTIONS

- Global variable must be started with capital letter.
- Local variable starts with small letters.
- Constant's name should have only capital letter.
- All functions, which encounters error condition, should either return 0 or 1 consistently.
- Code style should not contain any ambiguous things.
- Code should be well documented.
- Standard function/module will contain upto 10 lines of code.
- Don't use GOTO statements.

CODE REVIEW:

- This is cost effective and helps to find out the errors.
  - → Code Inspection → for finding human error
  - → Code Walkthrough → To find algorithmic/logical error.

Date _____

Page No _____

SOFTWARE DOCUMENTATION:

- **Internal:** It contains source code.
- **External:** It contains SRS, Installation documents, user manual, readme, design, test document.

Gunning Fog Index

- No. of years of formal education that is required to understand the meaning of the code.

$$\text{fog}(D) = 0.4 \times \left(\frac{\text{words}}{\text{sentences}}\right) + \left(\text{Percent of words having 3 or more syllabus.}\right)$$

Dt. 17/11/25

class 31

TESTING:

- Testing is done to see if the code is running properly or not.
- Testing is done to observe whether the code is working as expected or not.
- We are finding errors and bugs. (not fixing)
- There are 3 parts → (i) Identification of Module
  (ii) Generation of Test Cases
  (iii) Checking by running the test cases.

Date.
Page No.

Terminologies :

- **Mistakes.**

- **Errors :** It's a mistake

- **Failure :** A code is not giving desirable outputs.

- **Test Case :** Set of inputs given to module under test.
  It is a triplet ⟶ Input [I]
  State of Module [S]
  Desired Result [R]

ex: for USER PROFILE ⟶ name: _____
UID : _____
PW : _____
Email: _____
- - - - - -

test case :
[input : "Amit", "amit0101", "amit" .... ; State : "Input"
Result :]

- **Test suite :** Combination of all the test cases possible.

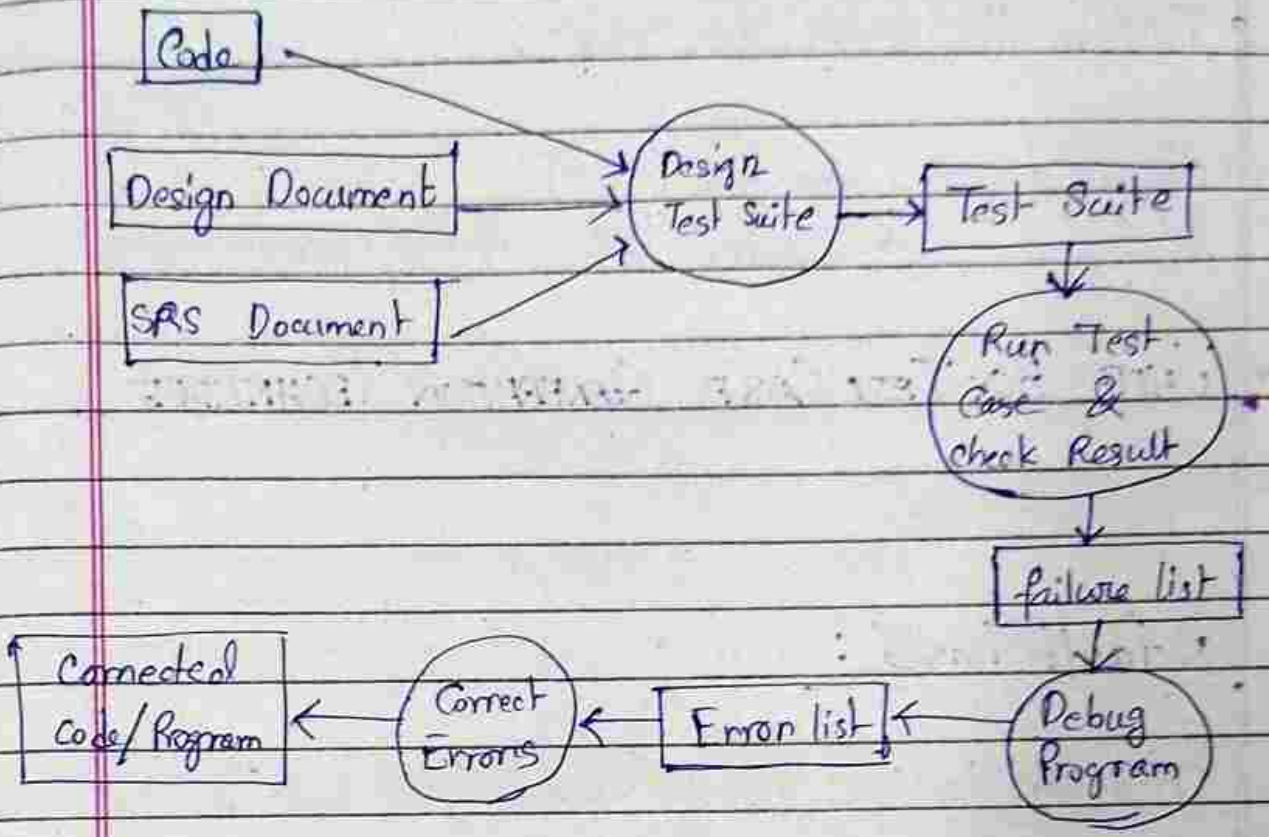
Verification vs Validation

- It confirms the output of specific modules.
- It validates that the SW meets the requirements.

Date. _____

Page No. _____

PROCESS OF TESTING



DESIGN OF TEST CASES:

(input & output of the module)

Black Box Approach: Understanding the behaviour of the module to generate test cases.

White Box Approach: Understanding the code of the module to design test case.

- Designing test case deals with defining valid and invalid data.

@ Equivalence Class Partitioning:

Data should be partitioned in 2 parts → valid class
→ invalid class

Date.
Page No.

ⓑ Boundary Value Analysis

- Finding the ~~boun~~ edge values.

ex: valid points are b/w [1,10]
↳ boundary values ⇒ (0,1,10,11)

- WHITE BOX TEST CASE GENERATION TECHNIQUE

→ Observe Code

ⓐ fault based:

①Mutation testing: Change to the original code is mutation.

ⓑ Coverage based:
The test cases designed should cover all the lines of the codes atleast once.

- A code can have — ① Isolated statements
② Conditional statements
③ Iterative statements

- A Test suite should cover all the lines.

Date.
Page No.

(I) Statement Coverage:

ex.

```
int x,y;
cin >> x >> y;
if (x>y) cout << x;
else cout << y;
```

test suit maybe: {(5,1), (1,5), (6,1), (1,6)}

↳ all statements are executed.

(II) Branch Coverage:

TS1 = {(5,1), (6,1)} → Does not cover all the branches. (of the conditionals).

TS2 = {(6,1), (1,6), (3,2), (2,3)}

↳ covers all the branches.

(III) Condition Coverage:

A && B → returns boolean

- for condition coverage, there should be test cases covering all possibilities of conditions.

(IV) Multiple condition Coverage:

A && B || C

Date.
Page No.

A	B	C	A&&B (C1)	C1 || C
T	T	T	T	T
T	T	F	T	T
T	F	T	F	T
T	F	F	F	F
F	T	T	F	T
F	T	F	F	F
F	F	T	F	T
F	F	F	F	F

ex. if (temp > 150 || temp < 50)
warningOn();

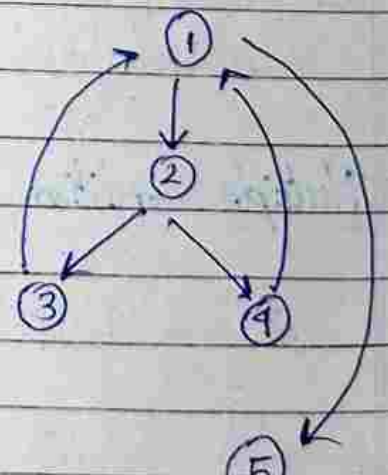
correction

for TS → {60, 200, 40}

→ [i/p : 40, state : execution, o/p : On]

PATH COVERAGE : (identifying isolated paths.)

ex.
```
int computeGCD (int x, int y)
{
  while (x != y) ①
  {
    if (x > y) ②
      x = x - y; ③
    else
      y = y - x; ④
  }
  return x; ⑤
}
```



Control Flow Graph

Date
Page No.

- You need to identify paths from code.
- Identify the isolated paths (unique path of execution of statements).

Types of Testing:

1. Unit Testing → Testing individual modules.
2. Integration Testing
3. System Testing:

   (A) Big Bang: Try to make a single interface and join all the module.

   (B) Bottom Up:

   (C) Top Down:

   (A) α-testing: Done by engineers.

   (B) β-testing: Known or choosen users/customers.

   (C) Acceptance test: All the users.